

DIGITAL SOUND SYNTHESIS

#06

array1



```
read -resize voice.wav array1
```

load sound file

```
soundfiler
```

```
62073
```

size of sound file

```
expr 44100 / $f1
```

```
phasor~
```

```
*~
```

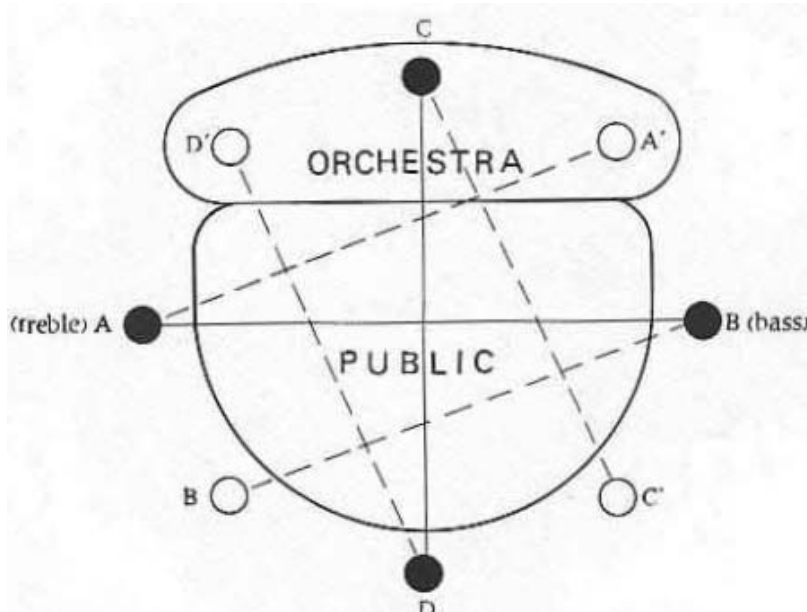
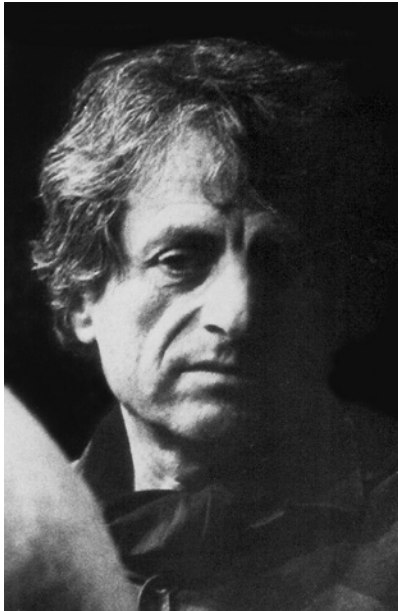
```
tabread4~ array1
```

```
dac~
```

GRANULAR SYNTHESIS

IANNIS XENAKIS

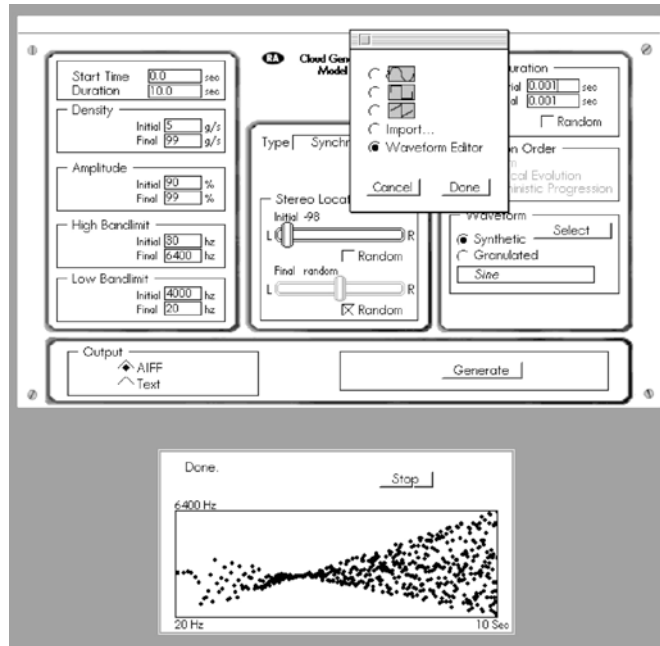
Early conceptual granular synthesis, tape-based compositions



<https://joshstovall.com/writing/granular-synthesis/#1.3>

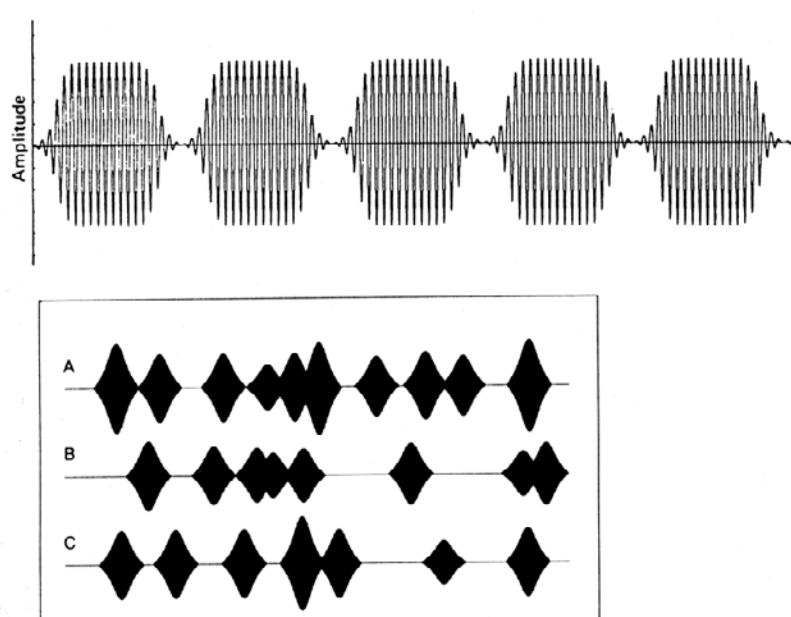
CURTIS ROADS

Digital granular synthesis & microsound theory



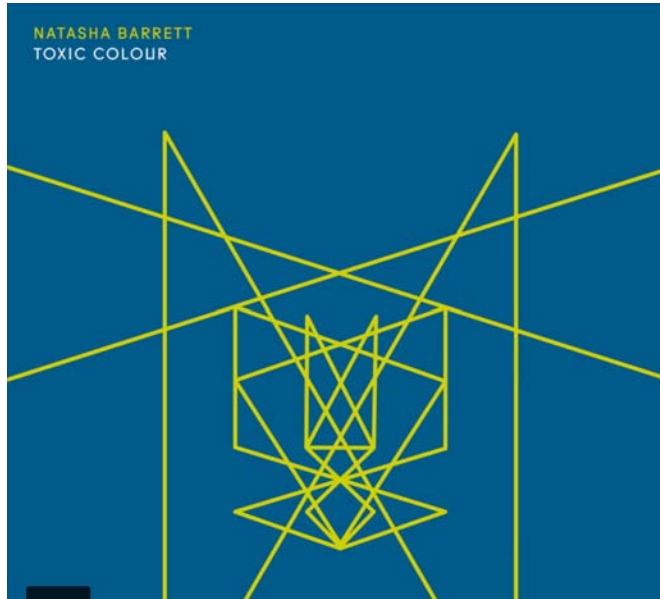
Barry Truax

Real-time granular synthesis, live implementation



NATASHA BARRETT

extending granular synthesis into immersive spatial audio



<https://www.15questions.net/interview/natasha-barrett-about-directions-electronic-music/page-1/>

Key Parameters

- **Grain size**

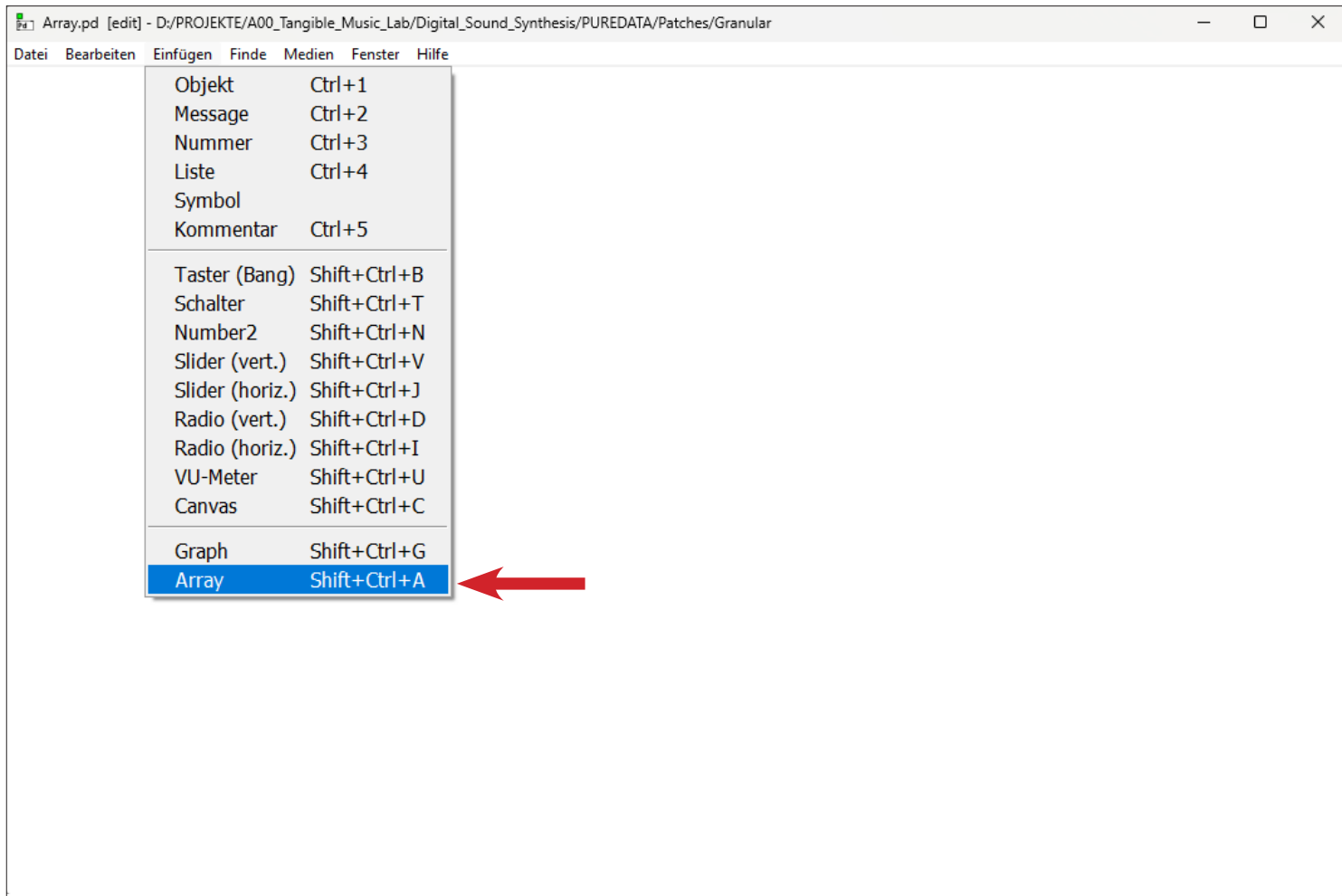
Duration of grains

- **Grain density**

Number of grains

- **Pitch**

- **Position**



Array-Eigenschaften

Array

Name: my_array

Größe: 100

☒ Inhalt speichern

Zeichne als:

☐ Polygon

☒ Punkte

☐ Bézier-Kurve

Array einfügen in:

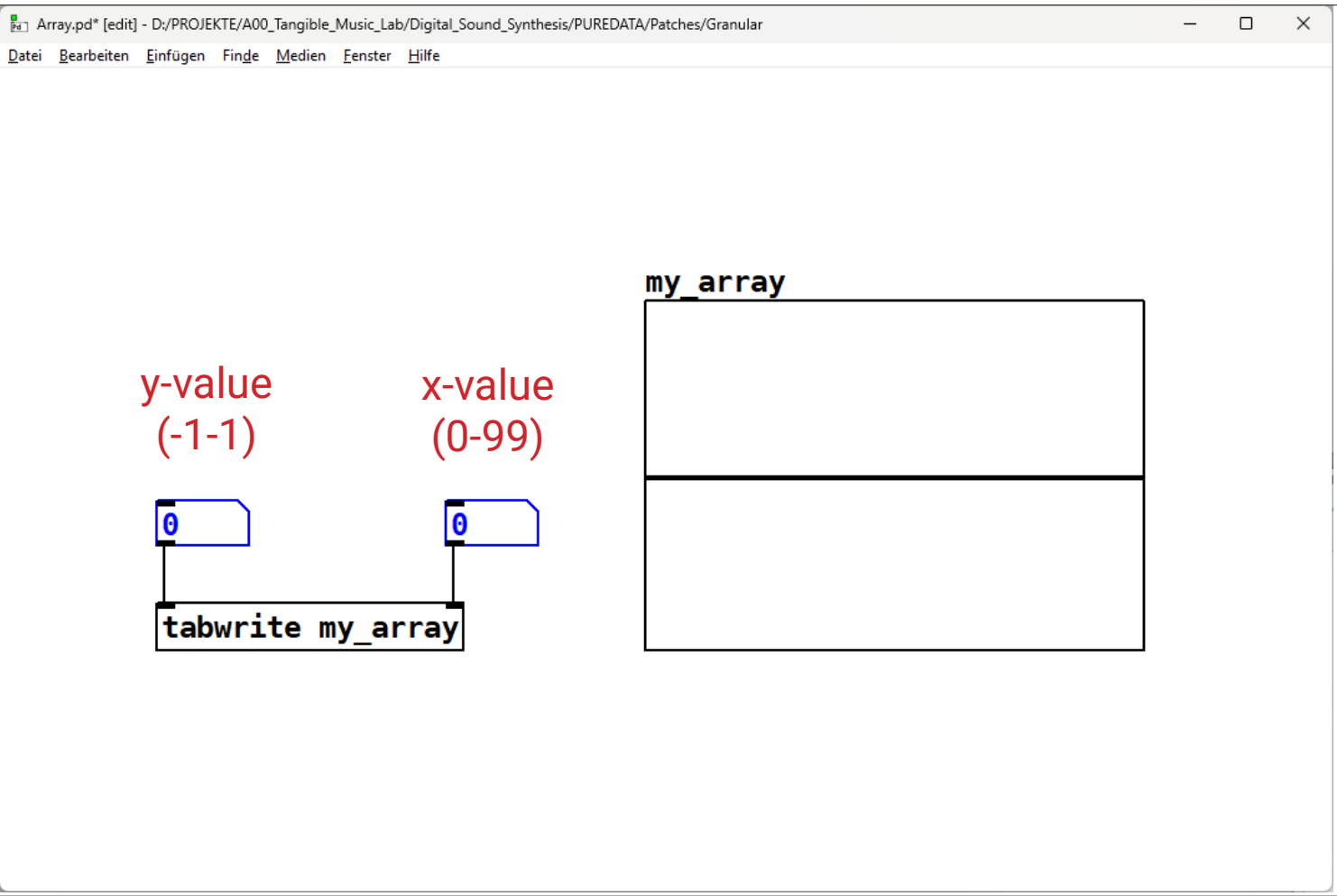
☒ neuen Graph

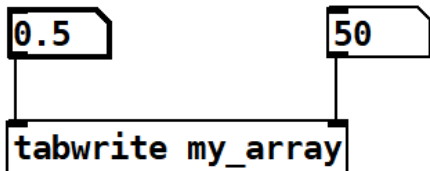
☐ letzten Graph

Abbrechen OK

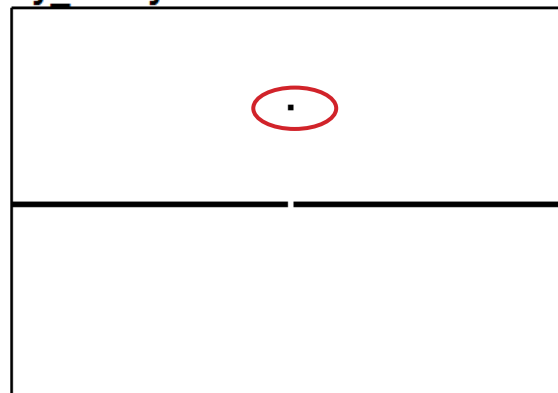
my_array

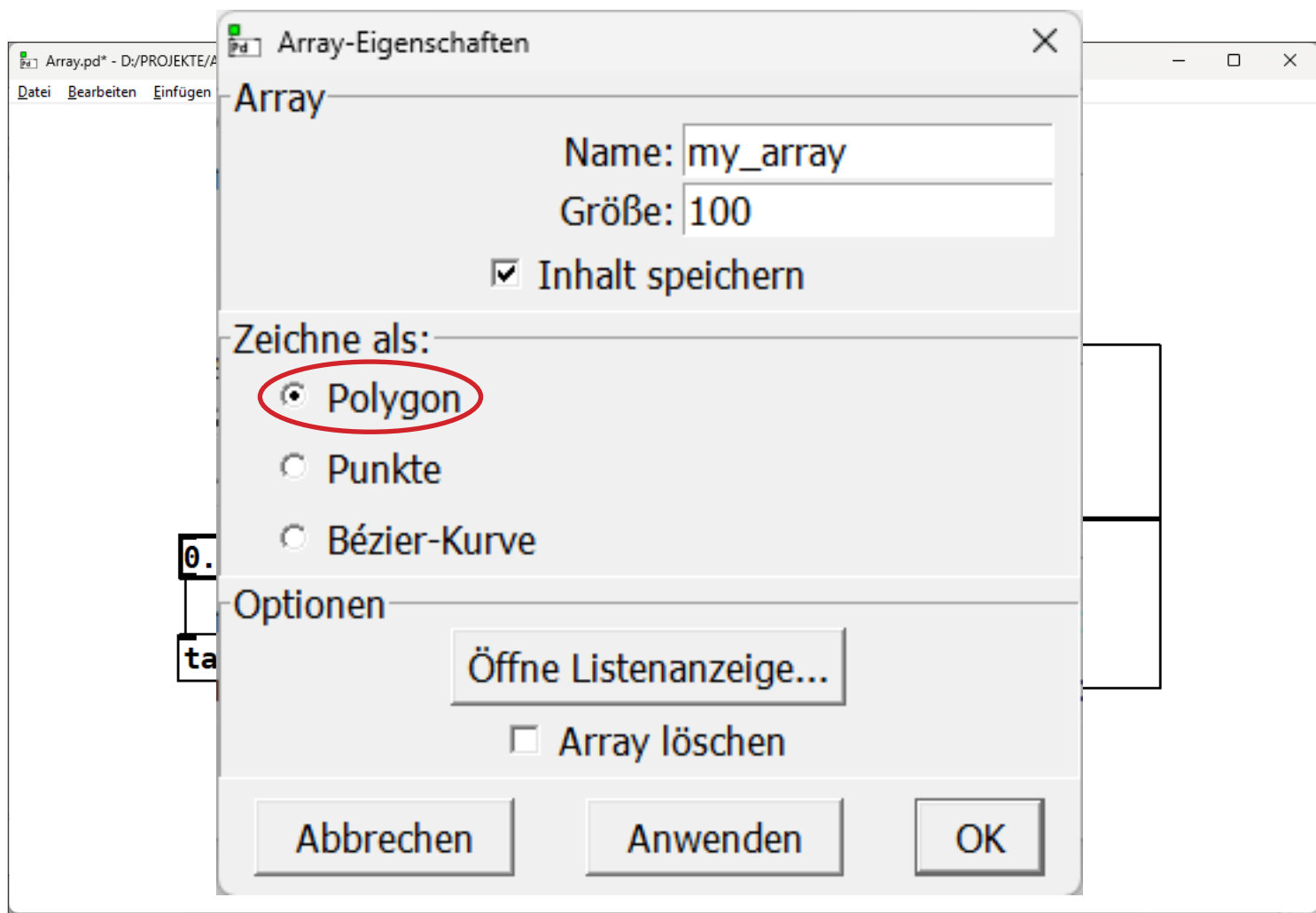






my_array



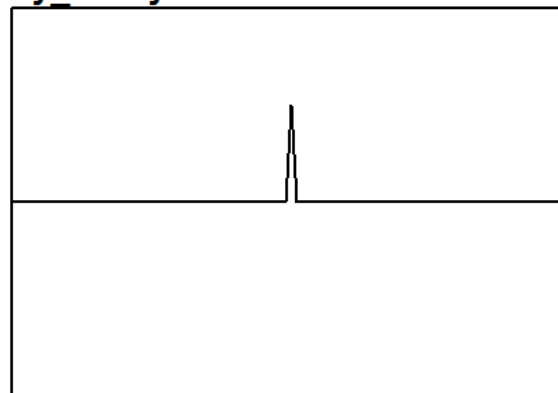


0.5

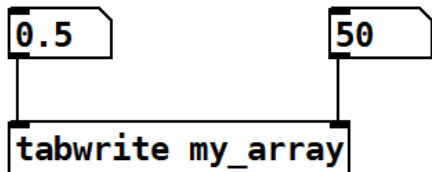
50

tabwrite my_array

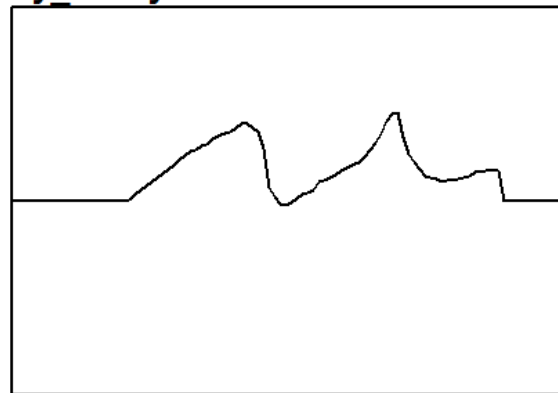
my_array



in "Execute-mode"
draw into array



my_array



set all values to 0

```
;
my_array const 0
```

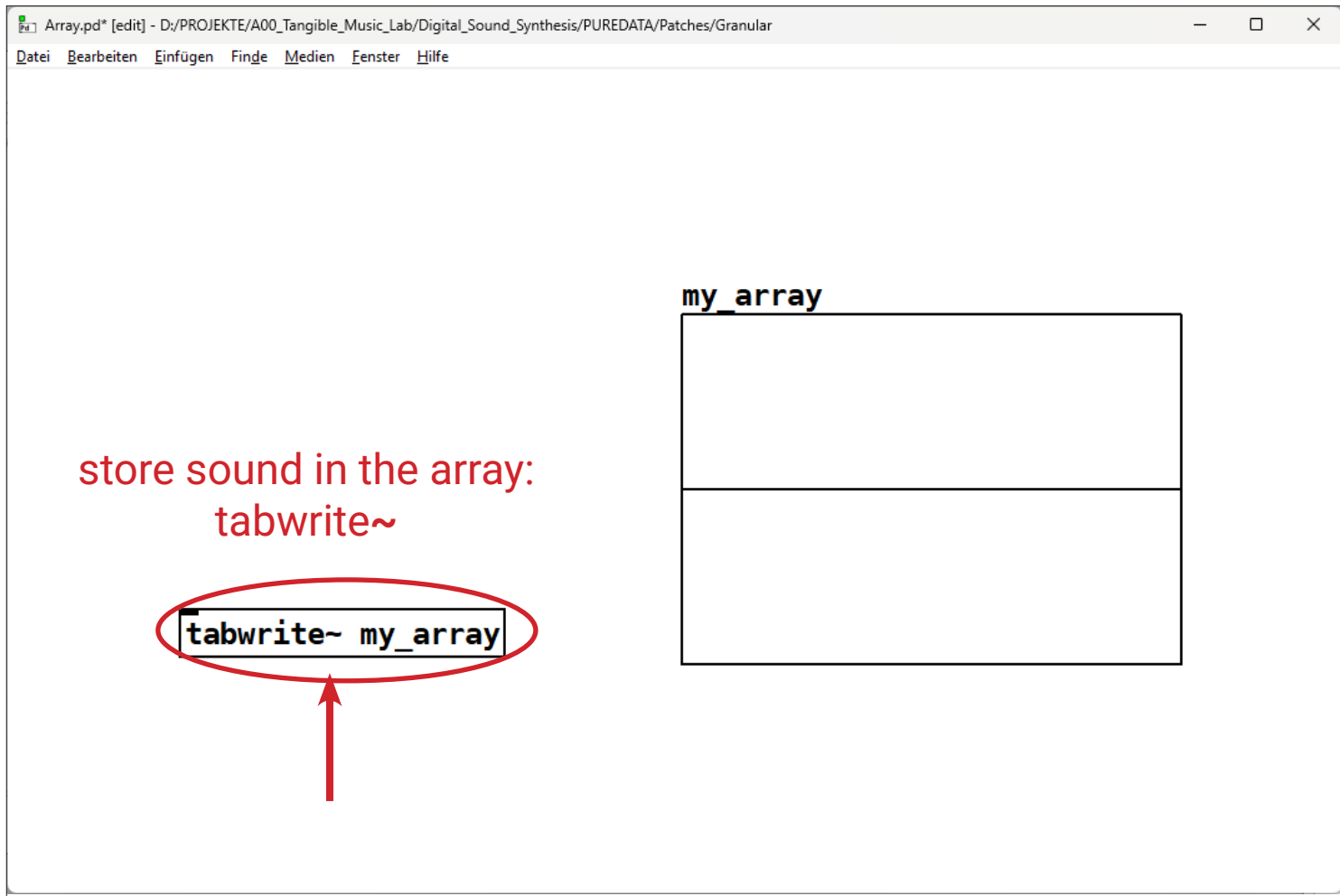
0.5

50

```
tabwrite my_array
```

my_array



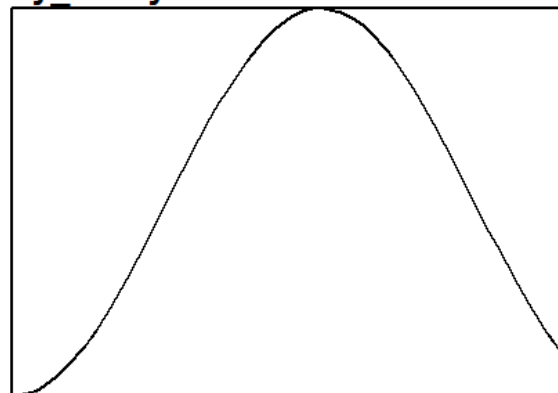


osc~ 400



tabwrite~ my_array

my_array



Record soundfile

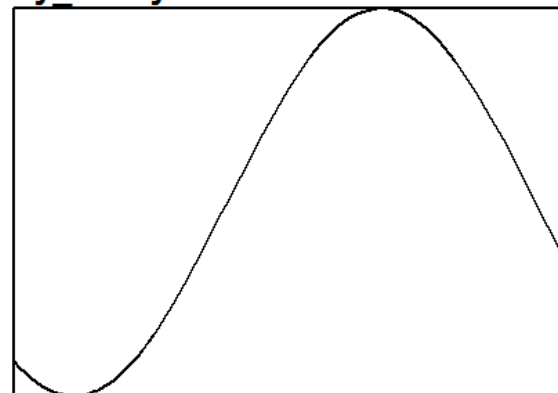
writesf~

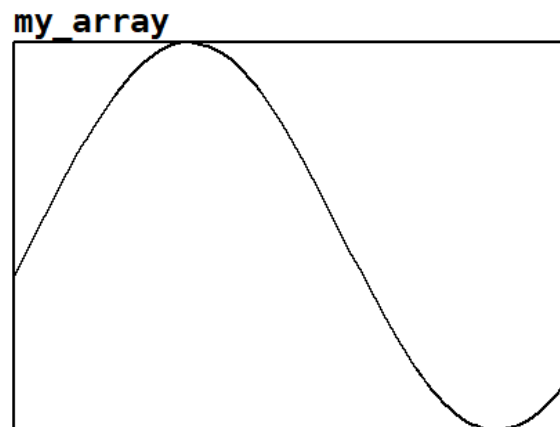
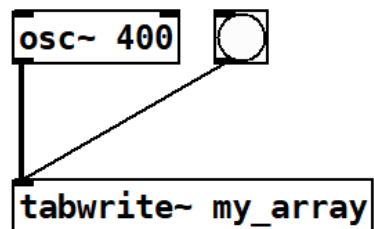
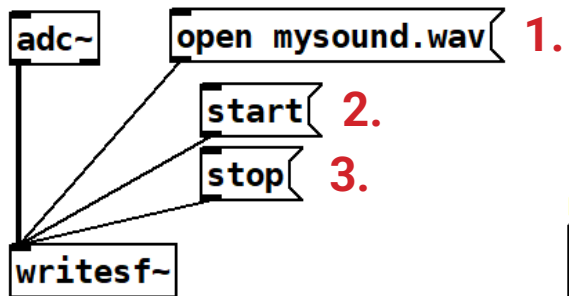
osc~ 400

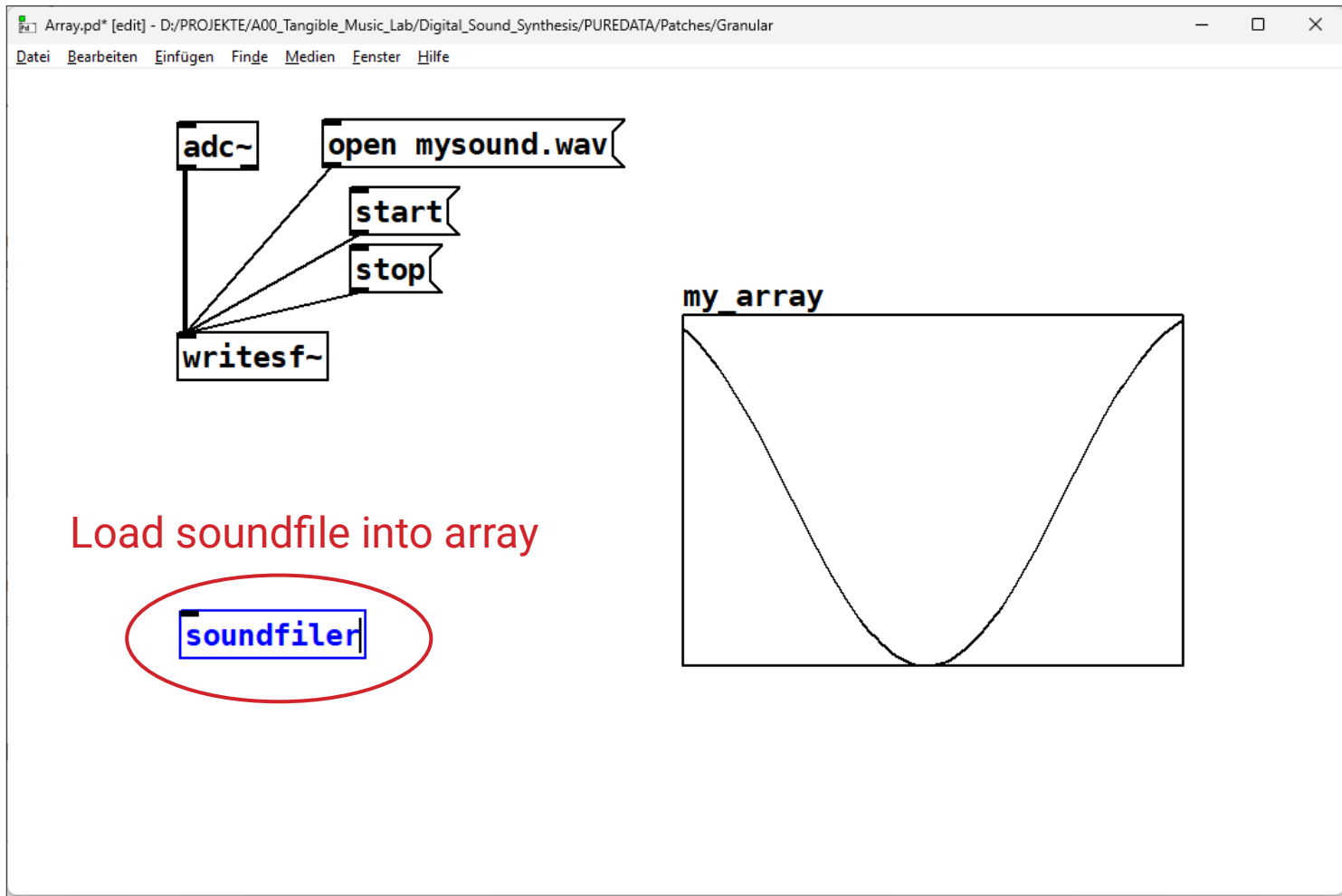


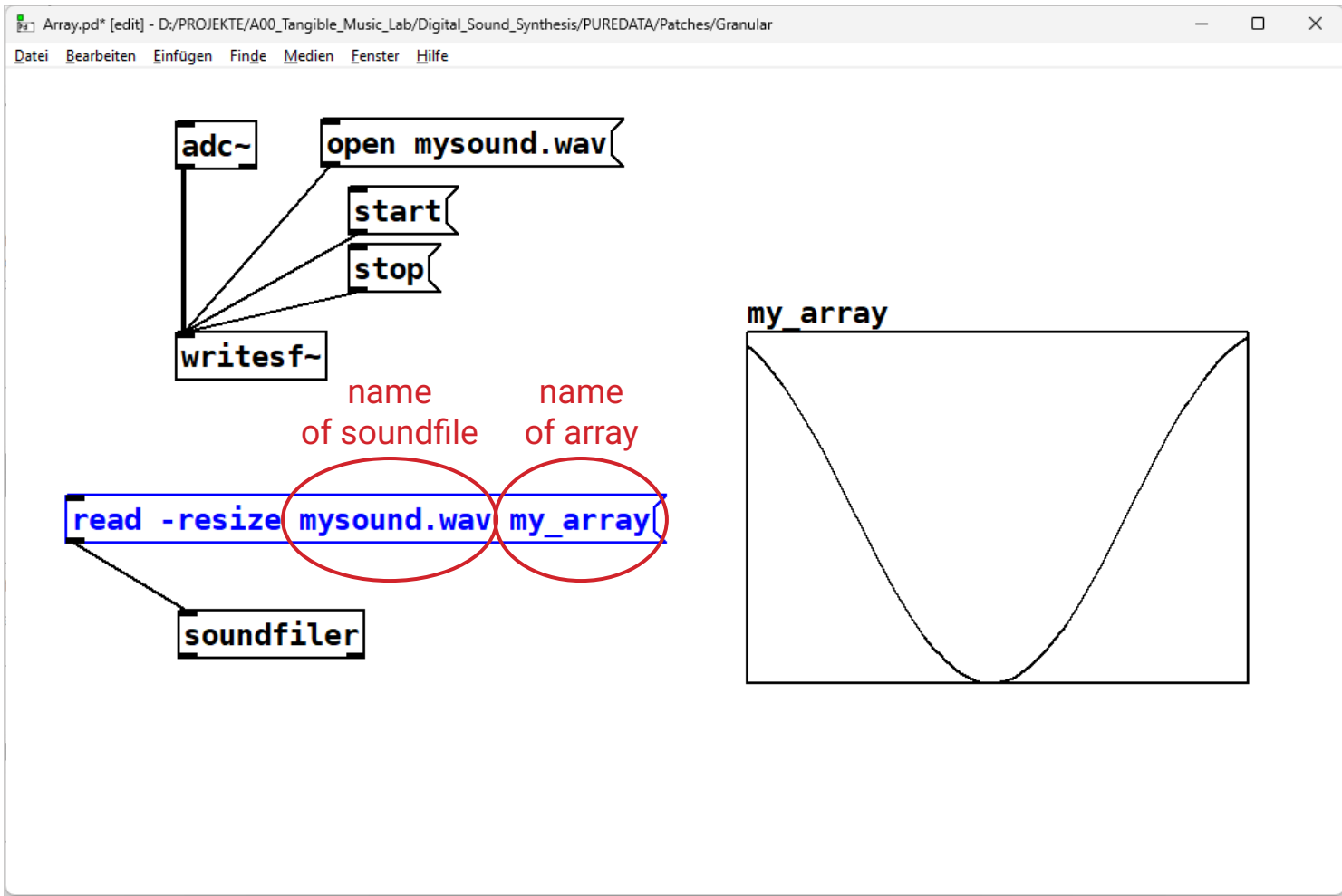
tabwrite~ my_array

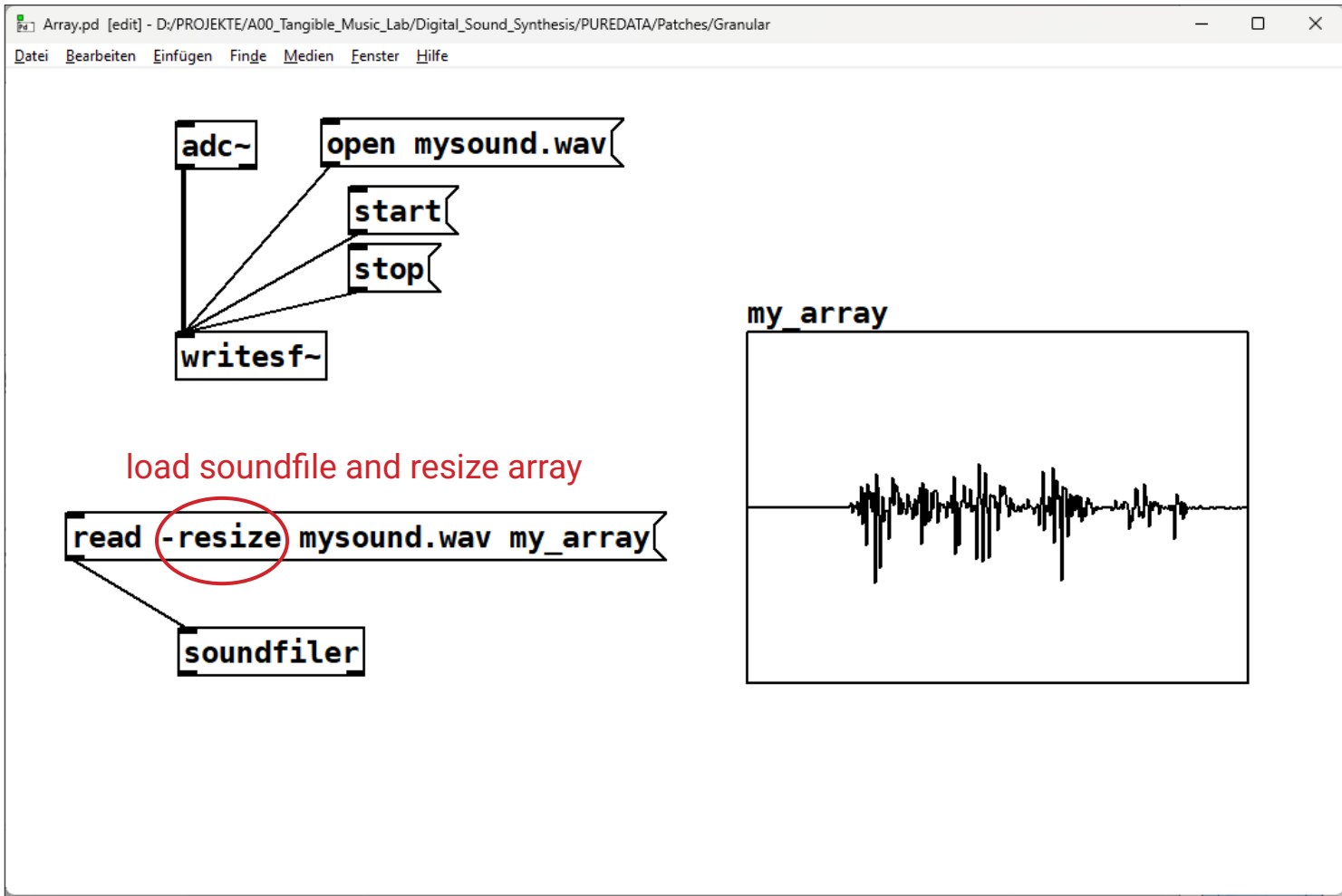
my_array











Array.pd* [edit] - D:/PROJEKTE/A00_Tangible_Music_Lab/Digital_Sound_Synthesis/PUREDATA/Patches/Granular

Datei Bearbeiten Einfügen Finde Medien Fenster Hilfe

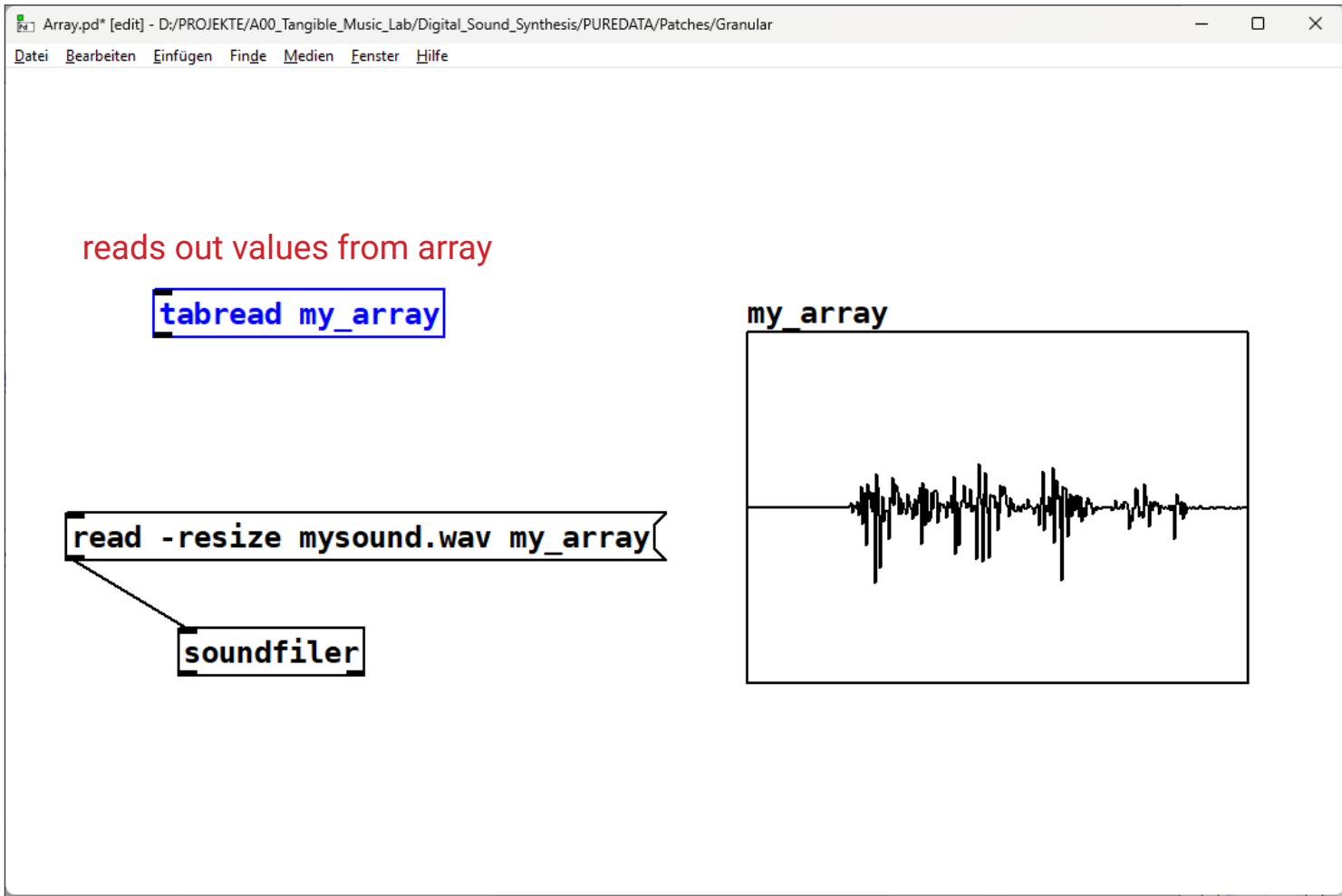
reads out values from array

`tabread my_array`

`read -resize mysound.wav my_array`

`soundfiler`

`my_array`



12547

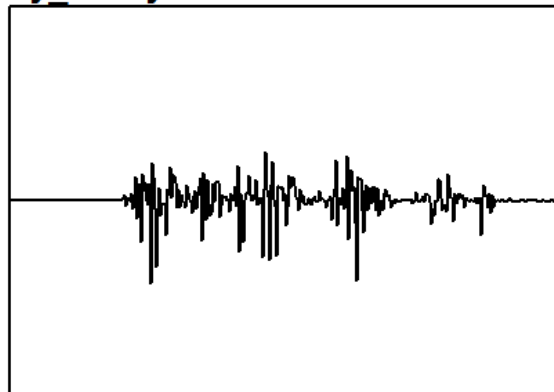
tabread my_array

0.001

read -resize mysound.wav my_array

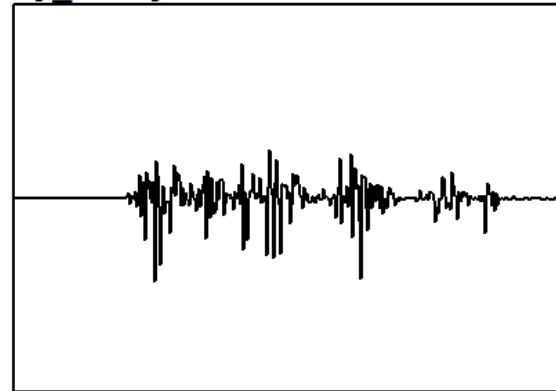
soundfiler

my_array



tabread~ my_array

my_array



read -resize mysound.wav my_array

soundfiler

reads out soundfile
from sample 0 to 114559
in 2600 milliseconds

0, 114559 2600

line~

tabread~ my_array

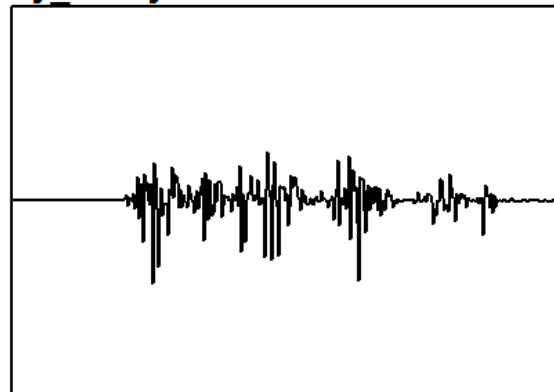
*~ 0.7

dac~

read -resize mysound.wav my_array

soundfiler

my_array



calculate duration:
samplesize (number of file samples) / **sample rate** (44100)

0, 114559 2600

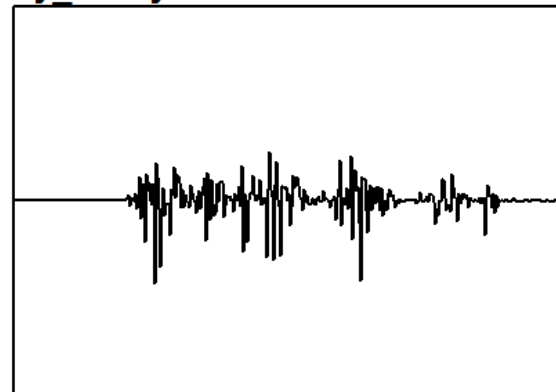
line~

tabread~ my_array

*~ 0.7

dac~

my_array



read -resize mysound.wav my_array

soundfiler

play it backward

114559, 0 2600

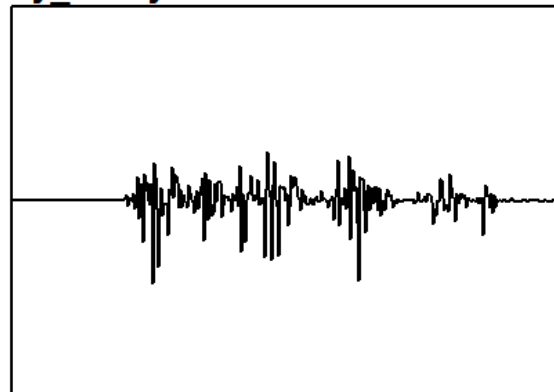
line~

tabread~ my_array

*~ 0.7

dac~

my_array



read -resize mysound.wav my_array

soundfiler

play it backward slower

114559, 0 5000

line~

tabread~ my_array

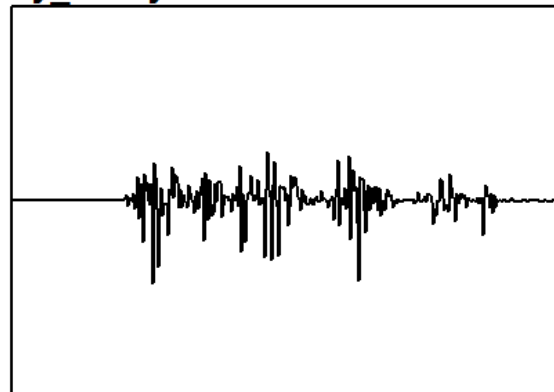
*~ 0.7

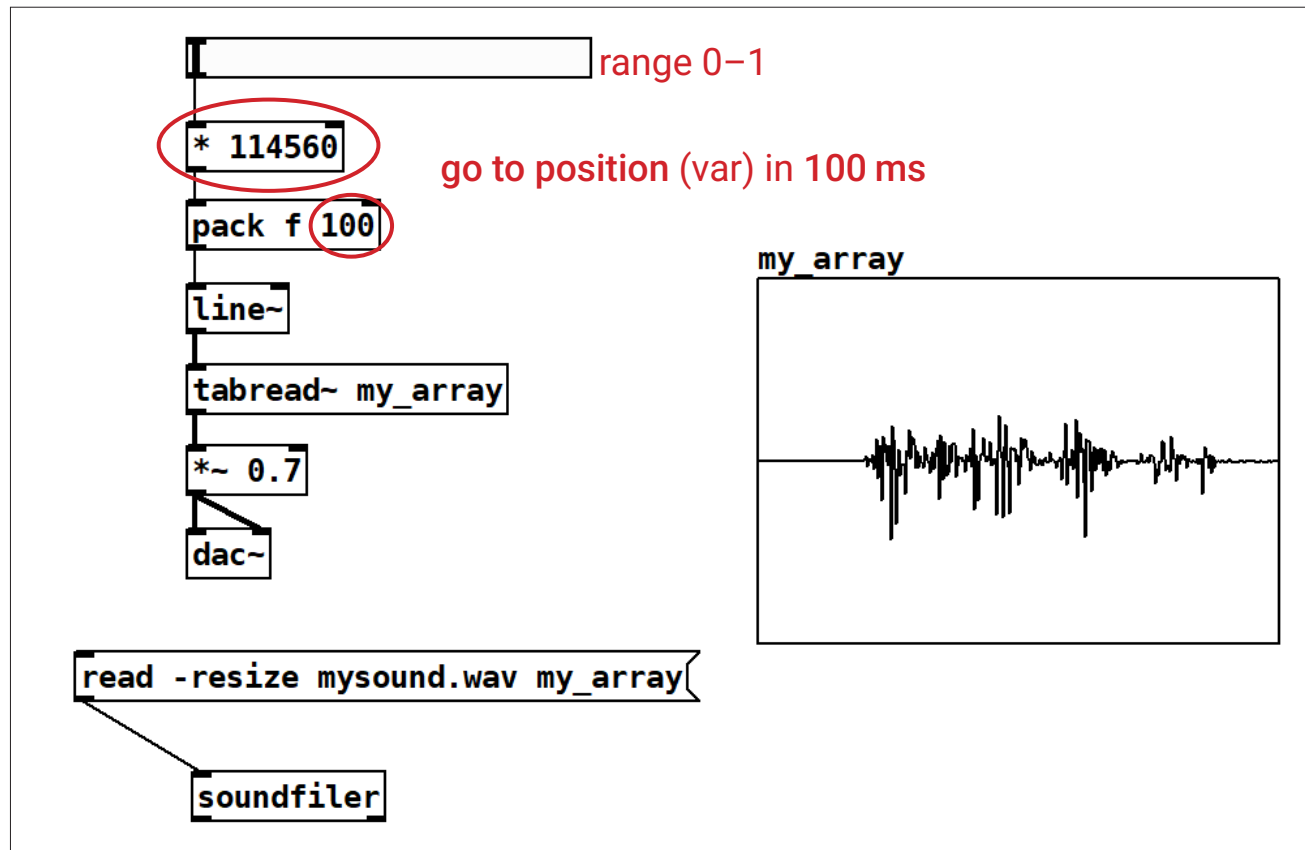
dac~

read -resize mysound.wav my_array

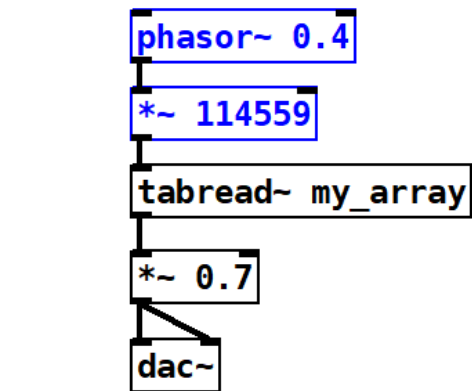
soundfiler

my_array





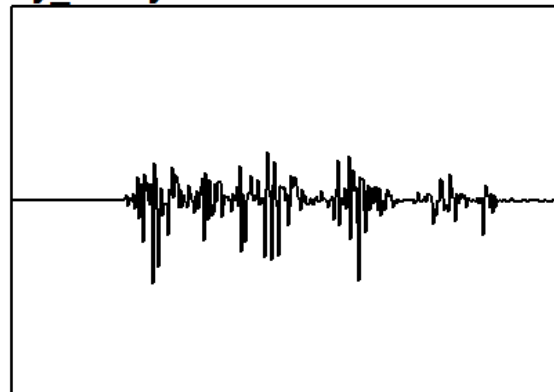
Use a phasor to play back the sample



read -resize mysound.wav my_array

soundfiler

my_array



Array.pd* [edit] - D:/PROJEKTE/A00_Tangible_Music_Lab/Digital_Sound_Synthesis/PUREDATA/Patches/Granular

Datei Bearbeiten Einfügen Finde Medien Fenster Hilfe

Calculate frequency:
44100 / sample size (= number of soundfile-samples)

phasor~ 0.38

*~ 114559

tabread~ my_array

*~ 0.7

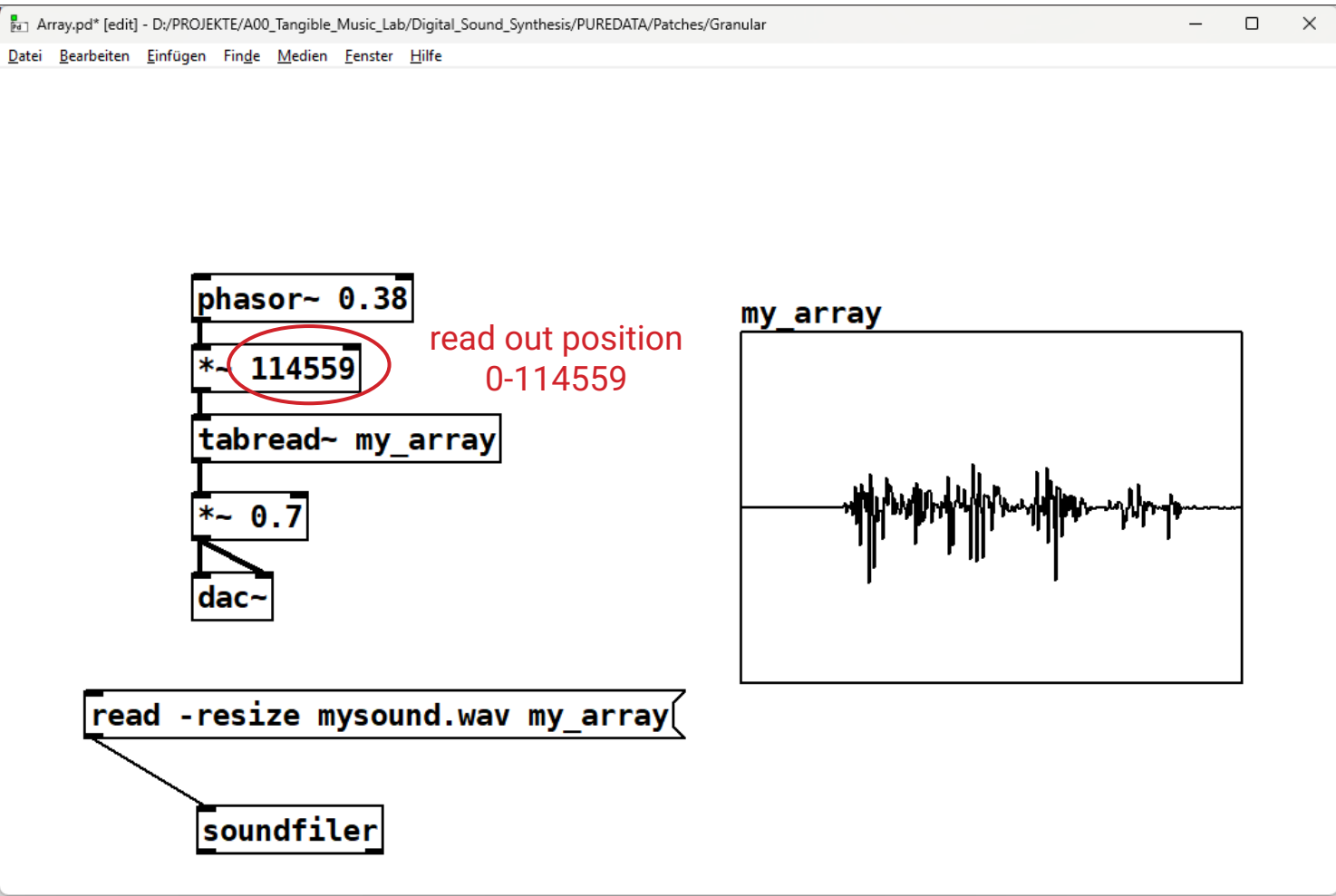
dac~

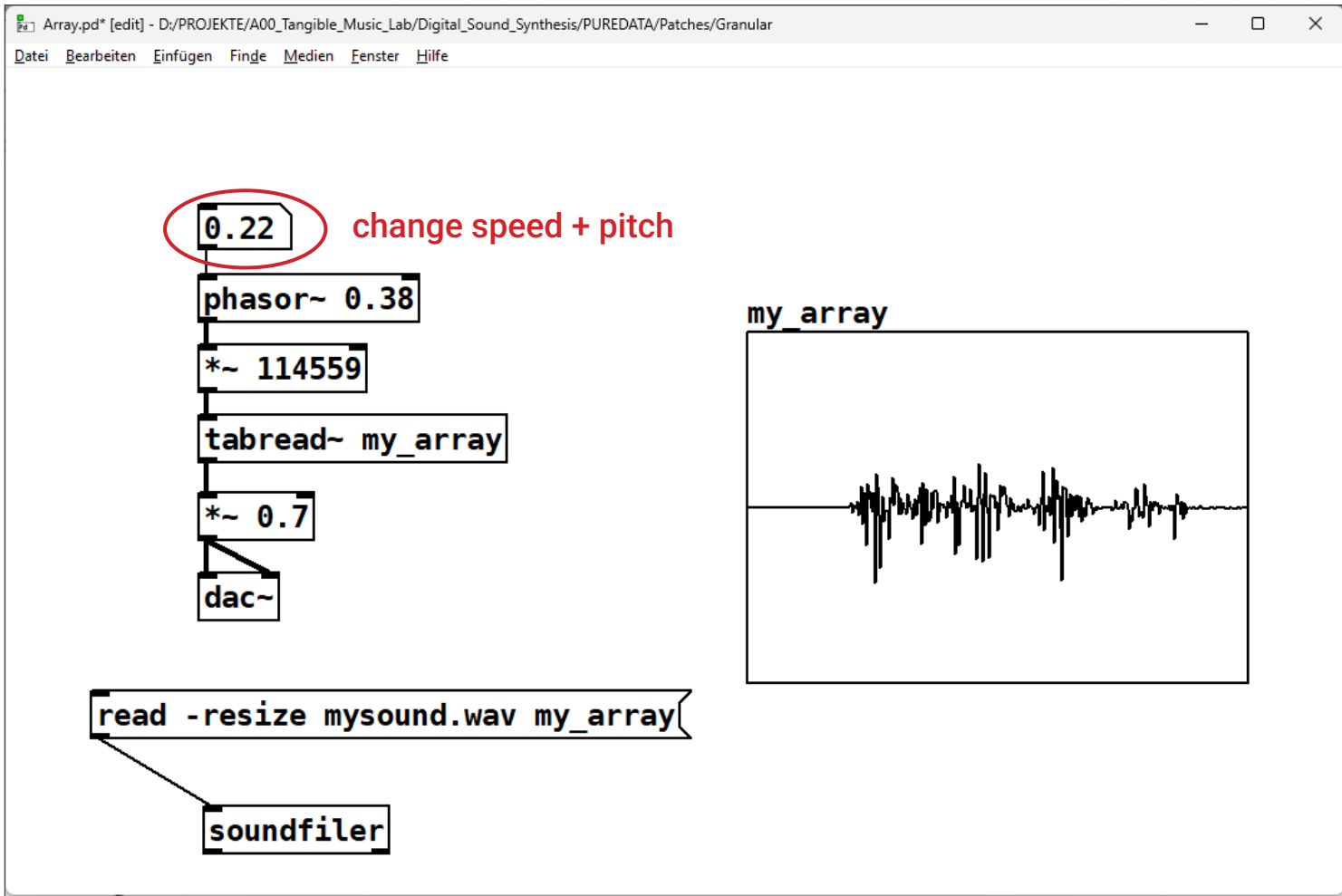
my_array

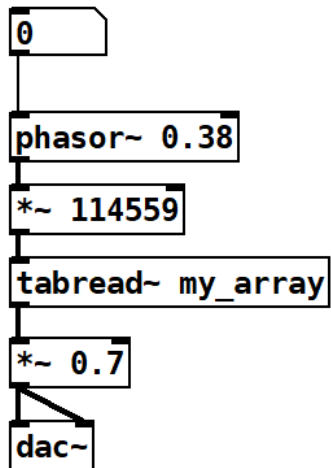
read -resize mysound.wav my_array

soundfiler

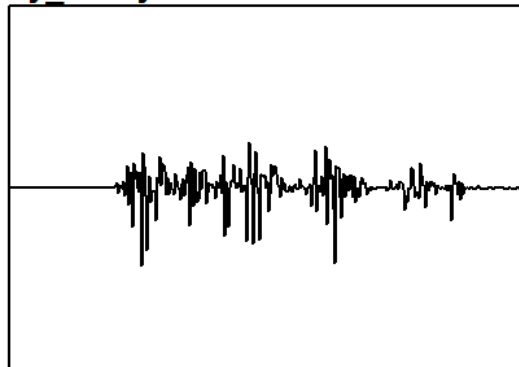
The screenshot shows a Pure Data patch window. The title bar reads 'Array.pd* [edit] - D:/PROJEKTE/A00_Tangible_Music_Lab/Digital_Sound_Synthesis/PUREDATA/Patches/Granular'. The menu bar includes 'Datei', 'Bearbeiten', 'Einfügen', 'Finde', 'Medien', 'Fenster', and 'Hilfe'. The patch area contains the following objects and connections: a 'soundfiler' object at the bottom left, connected by a line to a message box 'read -resize mysound.wav my_array'. The message box is connected to a 'tabread~ my_array' object. The output of 'tabread~' goes to a multiplication object '*~ 0.7'. The output of '*~ 0.7' goes to another multiplication object '*~ 114559'. The output of '*~ 114559' goes to a 'phasor~ 0.38' object, which is circled in red. The output of 'phasor~' goes to a 'dac~' object. To the right of the patch is a waveform display labeled 'my_array' showing a complex, noisy signal.







my_array



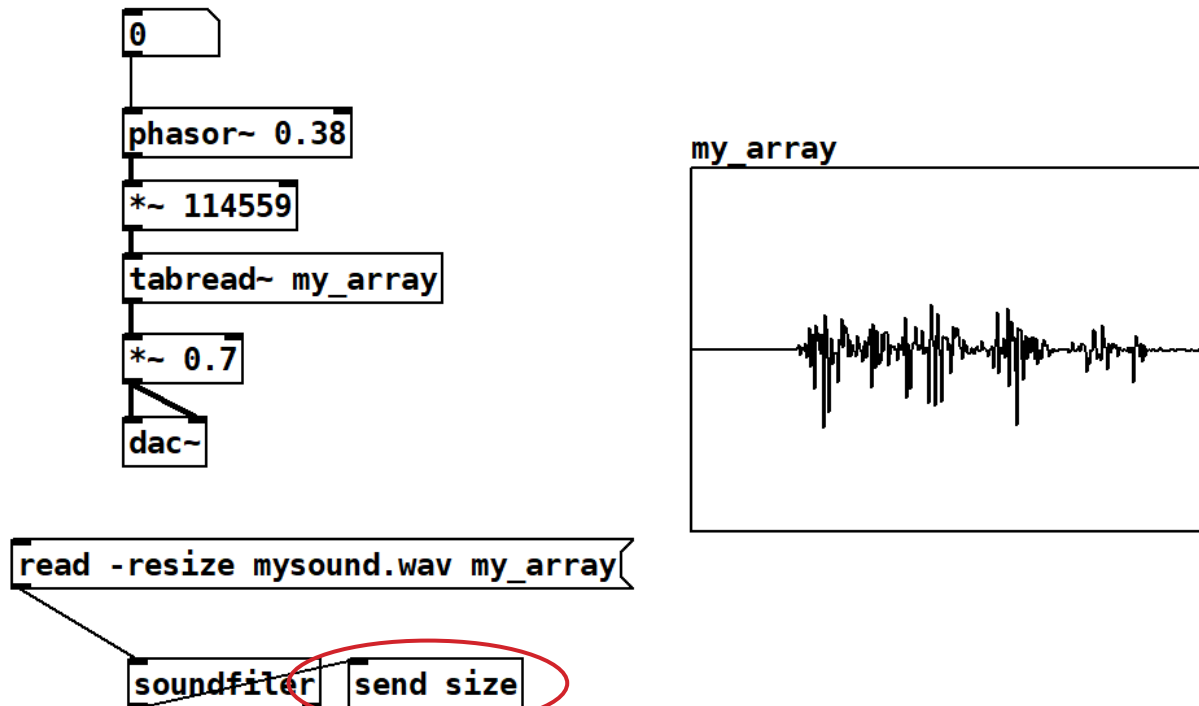
click

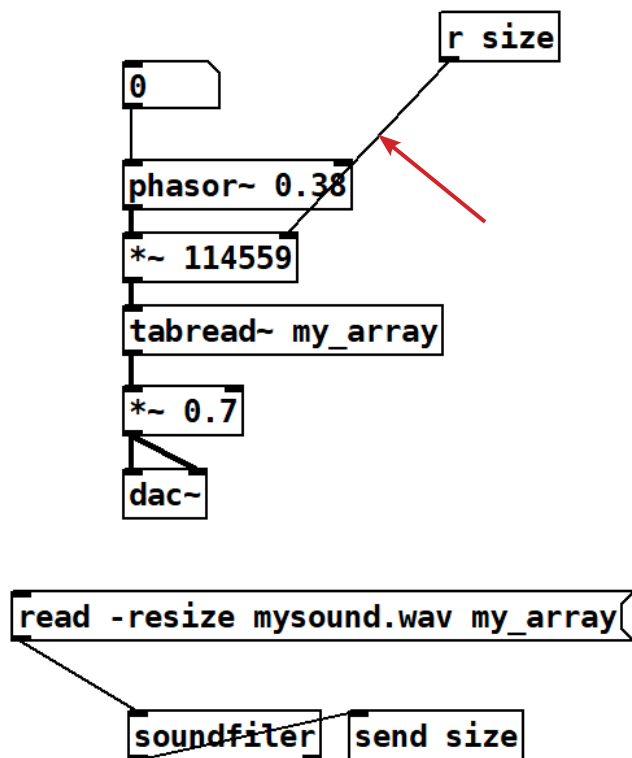
read -resize mysound.wav my_array

soundfiler 114560

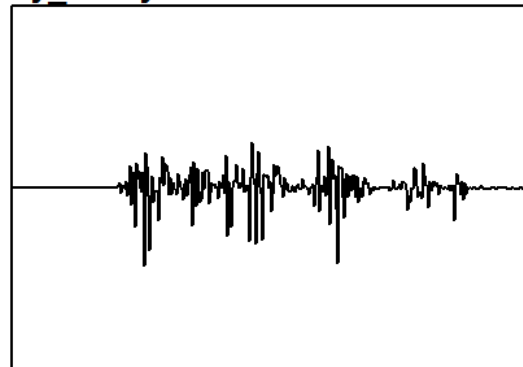
Pd Granular_phasor.pd* [edit] - C:/Users/ullar/.00A_PROJEKTE/A00_Tan...

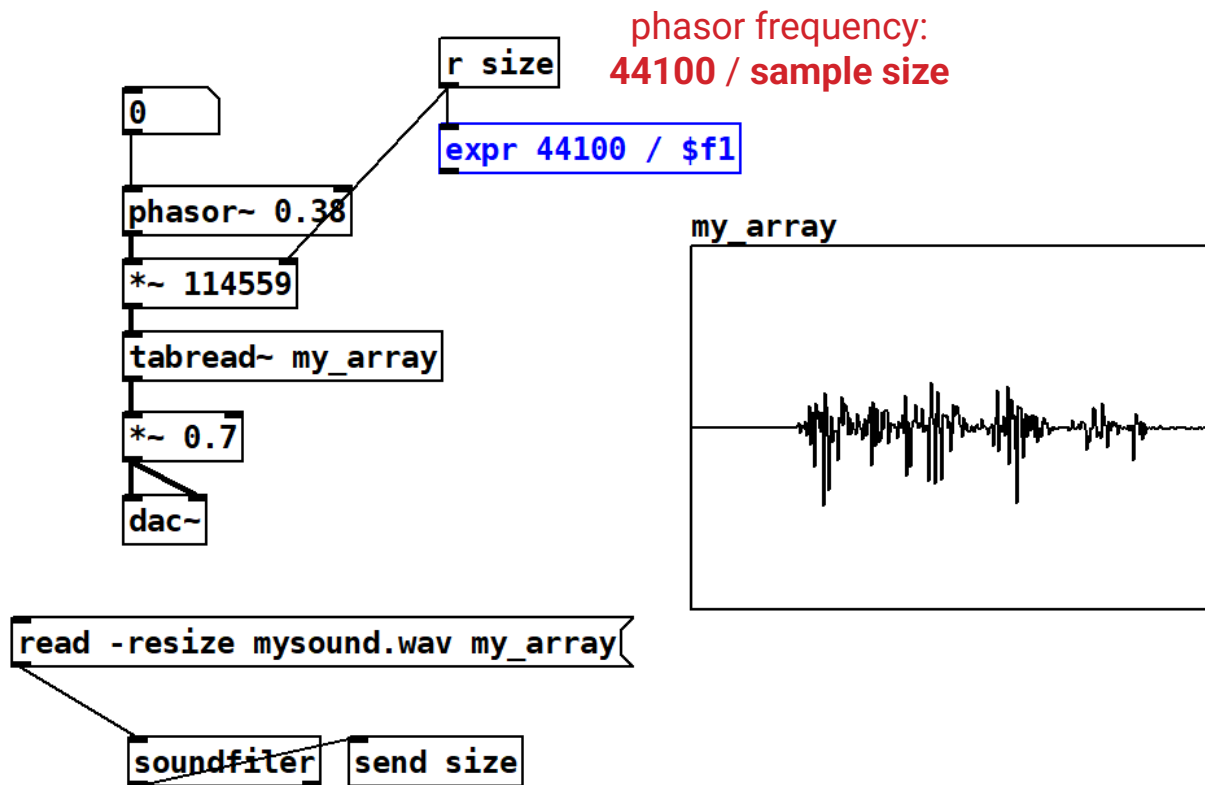
Datei Bearbeiten Einfügen Finde Medien Fenster Hilfe

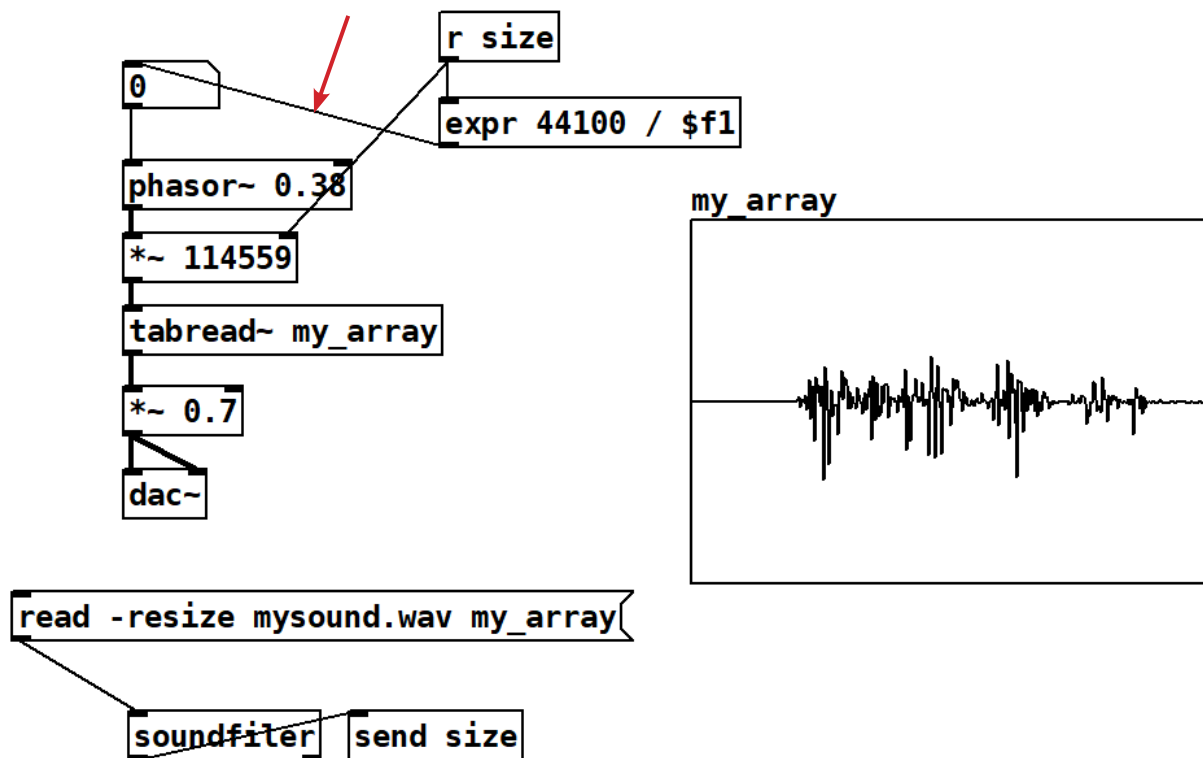


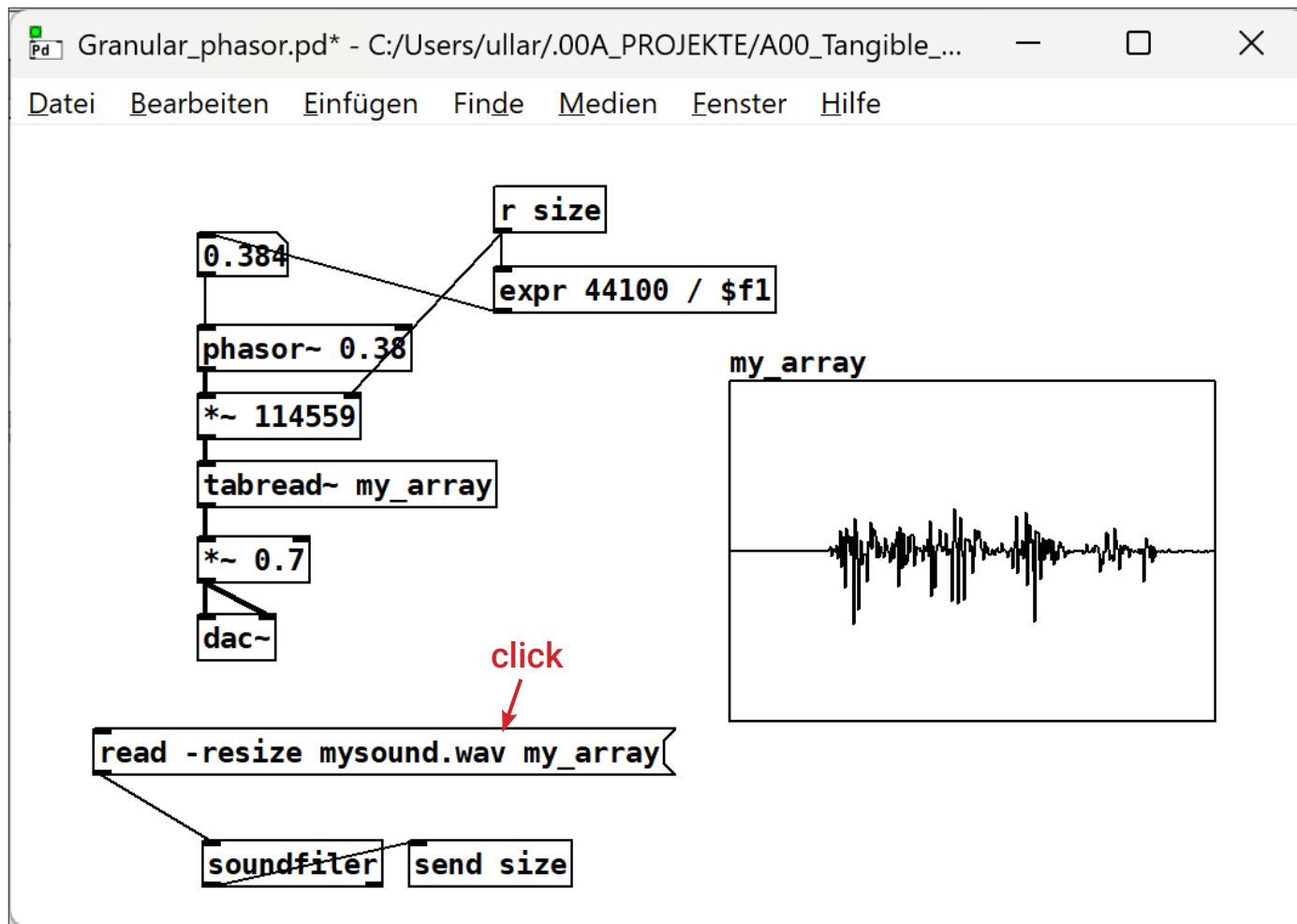


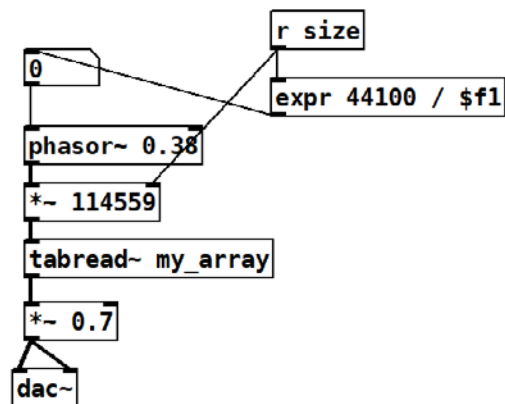
my_array



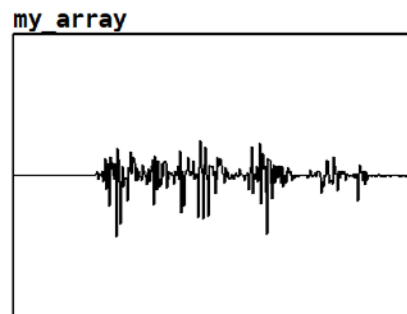
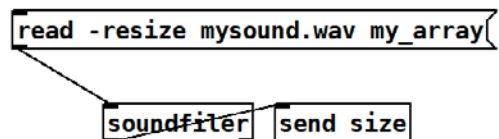


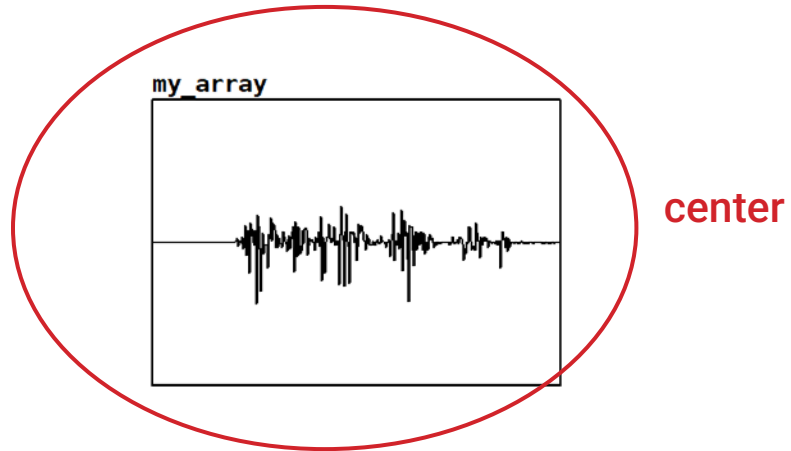
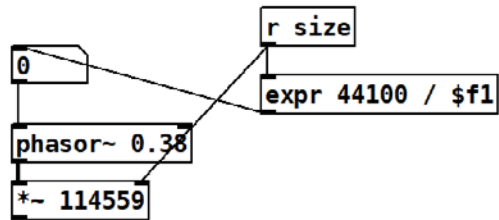




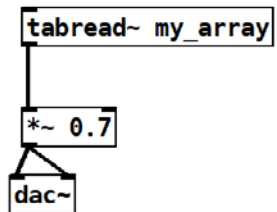


Building a sampler

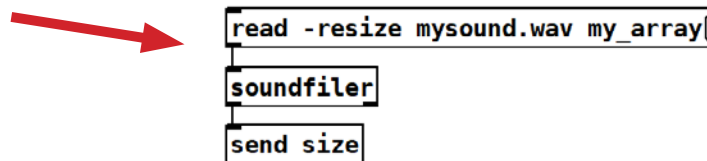


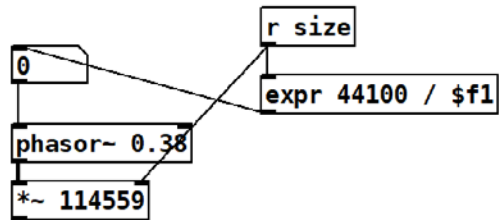


move

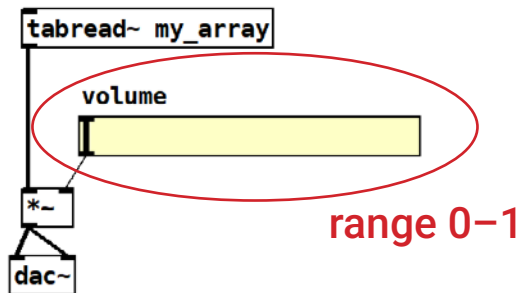
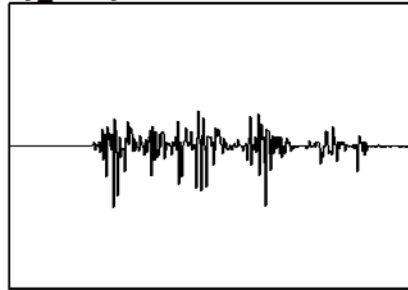


move



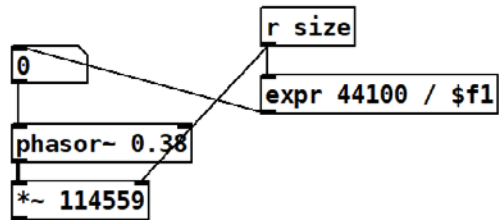


my_array

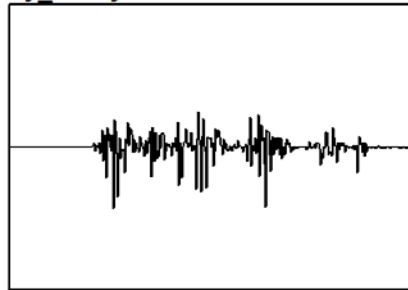


```

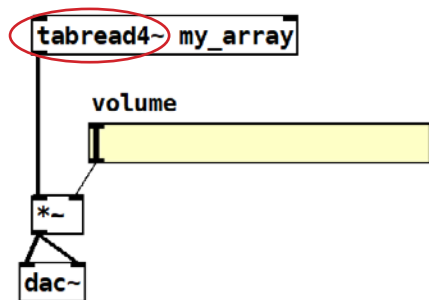
read -resize mysound.wav my_array
soundfiler
send size
  
```



my_array



change to **tabread4~**



```

read -resize mysound.wav my_array
soundfiler
send size

```

r size

expr 44100 / \$f1

phasor~ 0.38

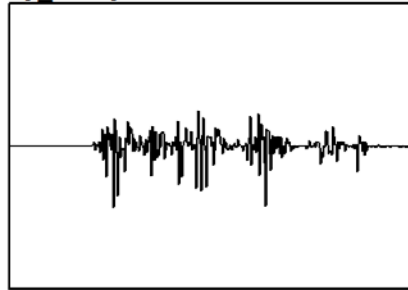
tabread4~ my_array

volume

*~

dac~

my_array



read -resize mysound.wav my_array

soundfiler

send size

r size

chunk size

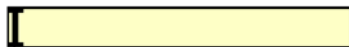


expr 44100 / \$f1

phasor~ 0.38

tabread4~ my_array

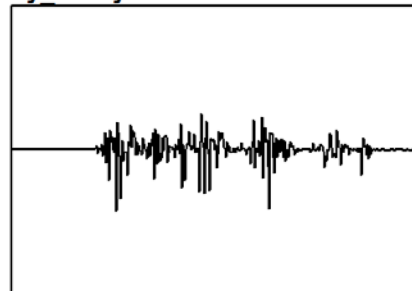
volume



*~

dac~

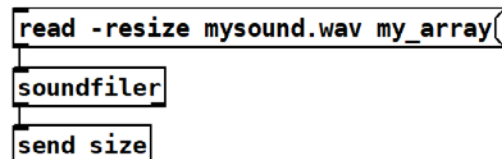
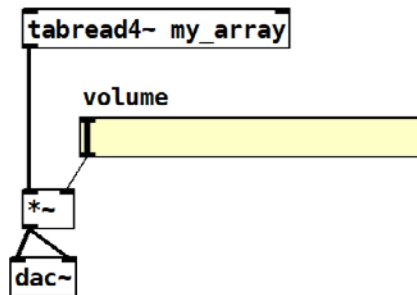
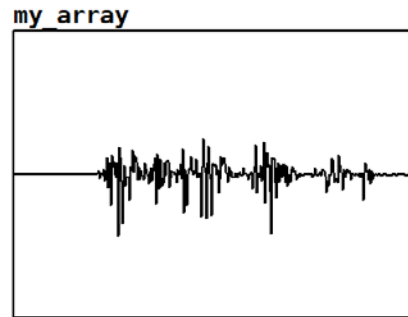
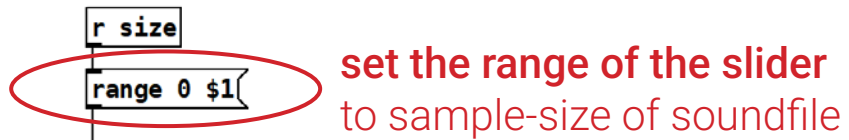
my_array

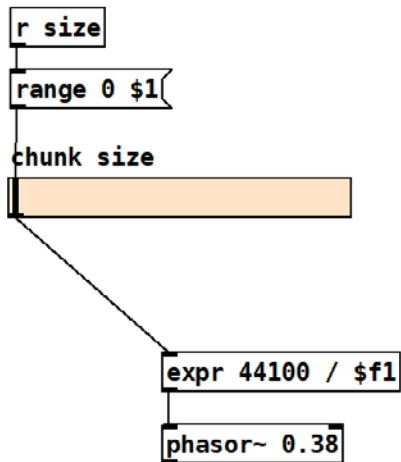


read -resize mysound.wav my_array

soundfiler

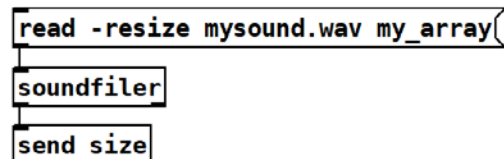
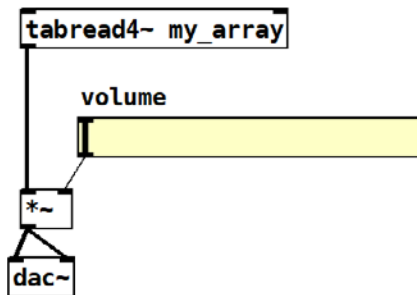
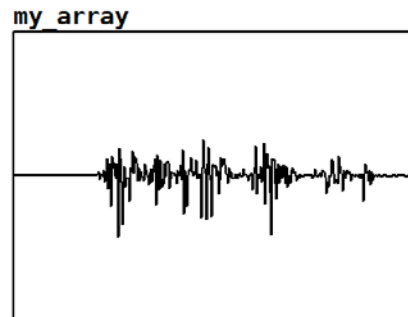
send size

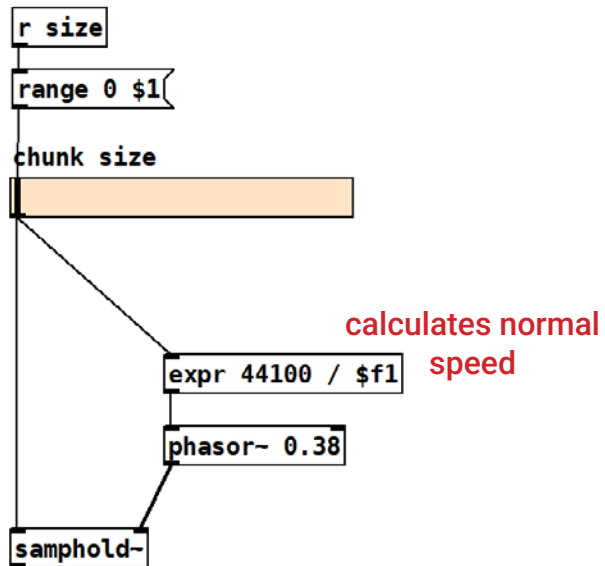




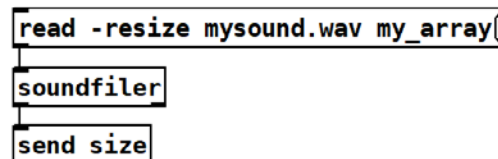
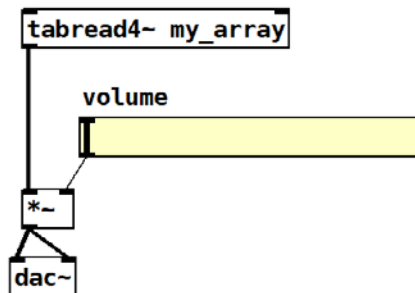
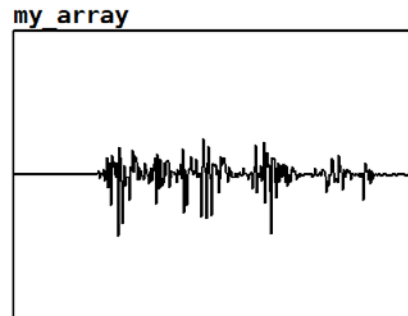
samphold~

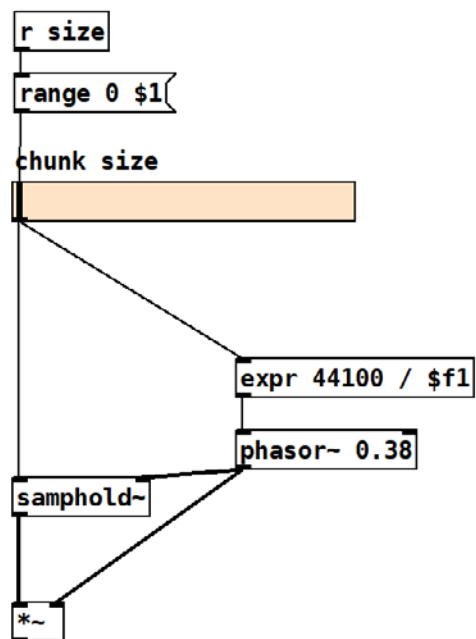
samples left input
when right input decreases



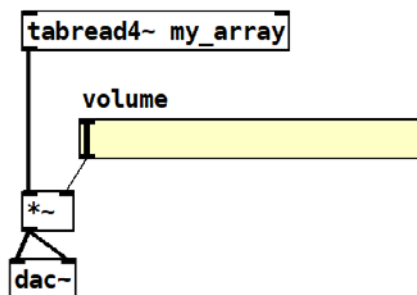
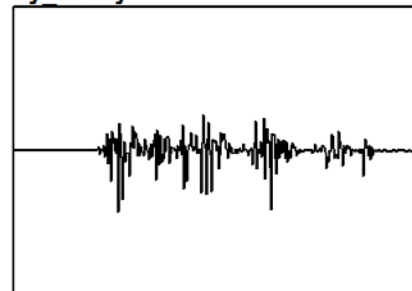


outputs chunksize
whenever phasor jumps to 0

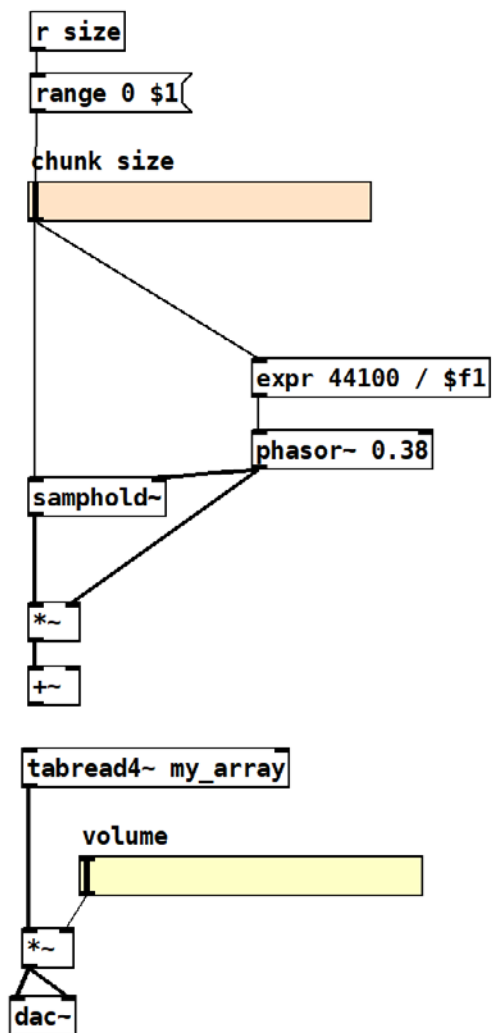




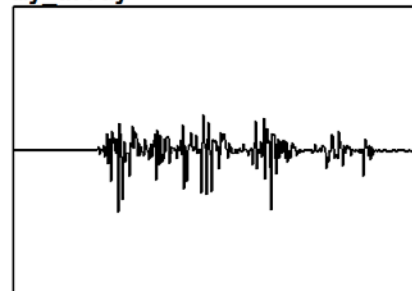
my_array



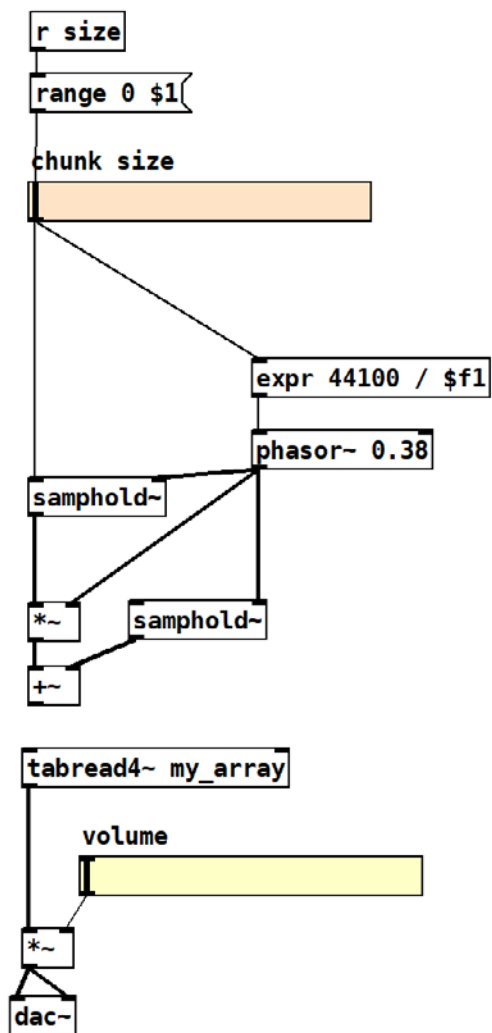
```
read -resize mysound.wav my_array  
soundfiler  
send size
```



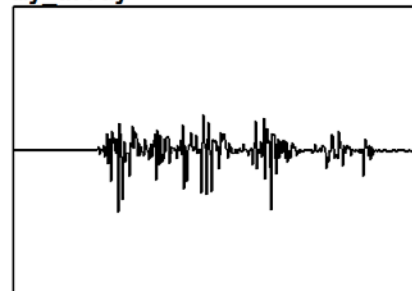
my_array



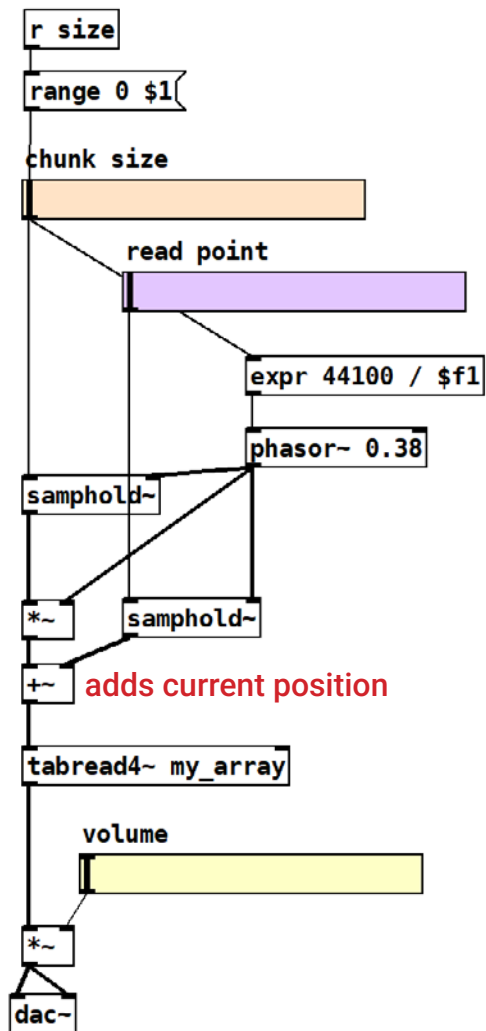
```
read -resize mysound.wav my_array  
soundfiler  
send size
```



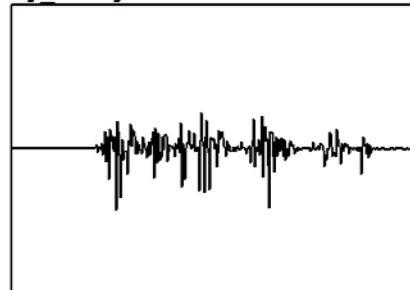
my_array



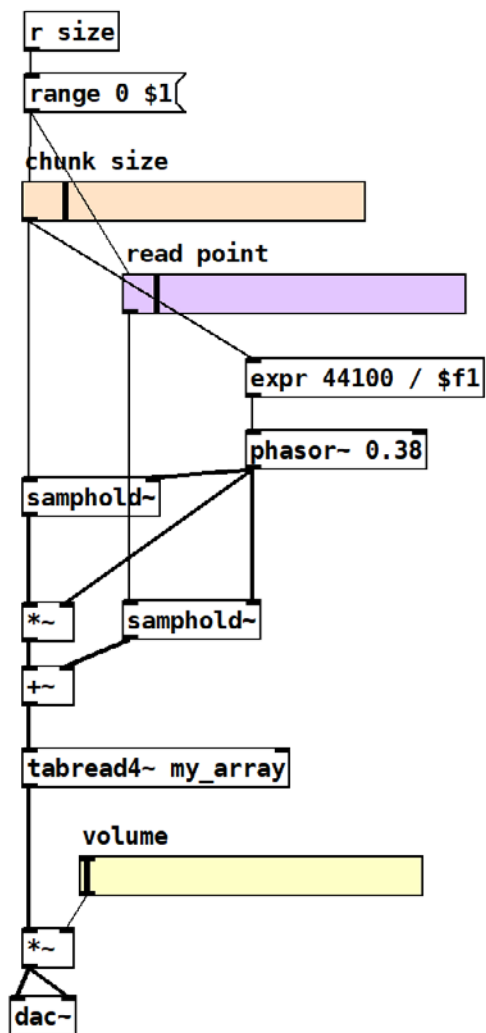
```
read -resize mysound.wav my_array  
soundfiler  
send size
```



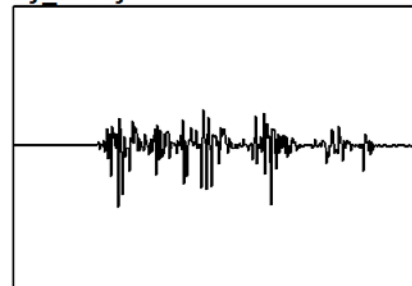
my_array



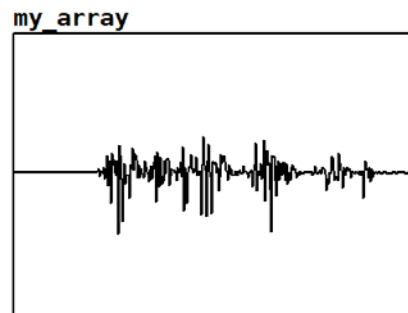
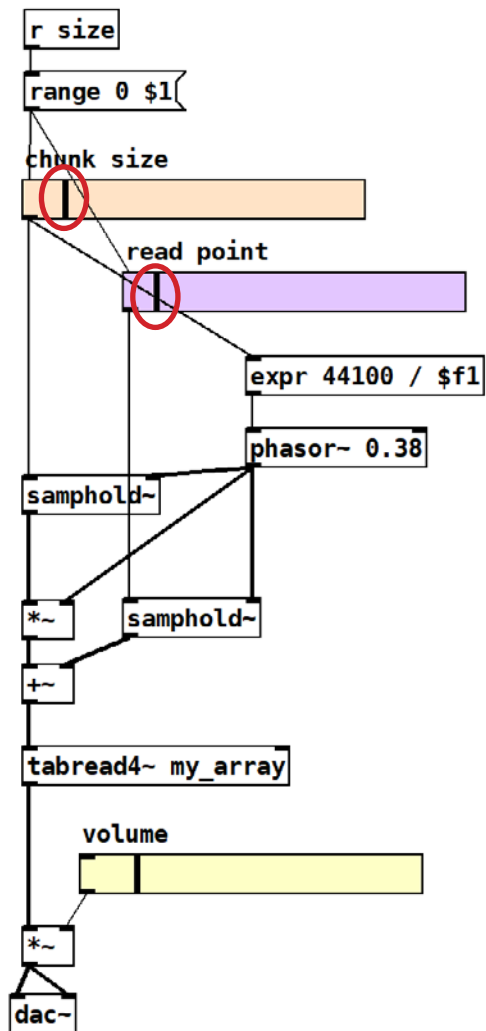
```
read -resize mysound.wav my_array  
soundfiler  
send size
```



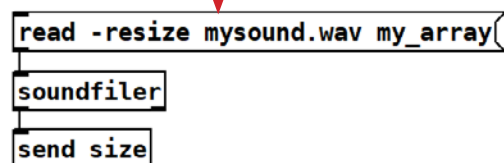
my_array

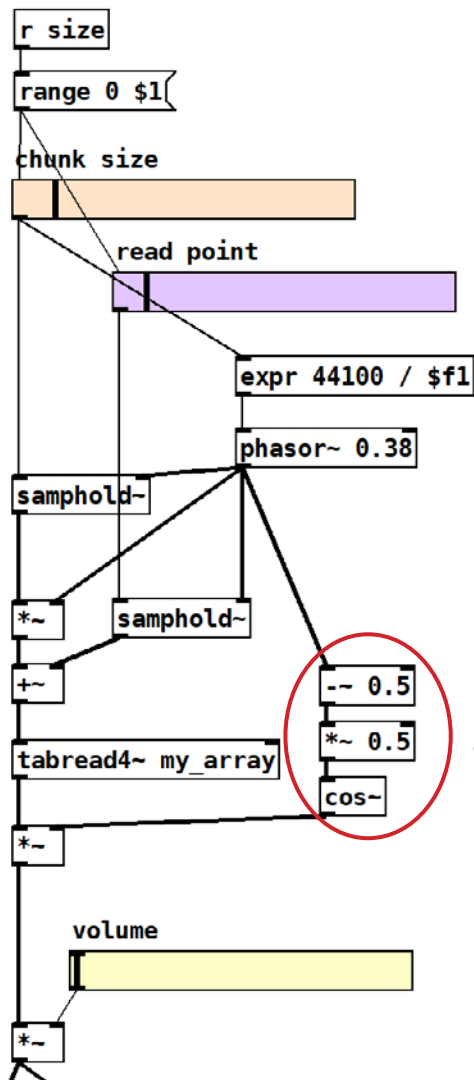


```
read -resize mysound.wav my_array  
soundfiler  
send size
```



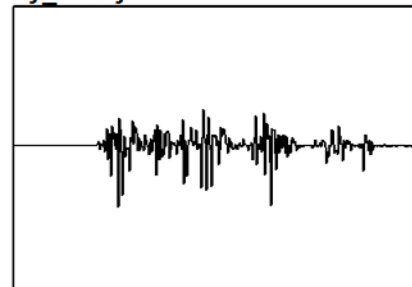
click!



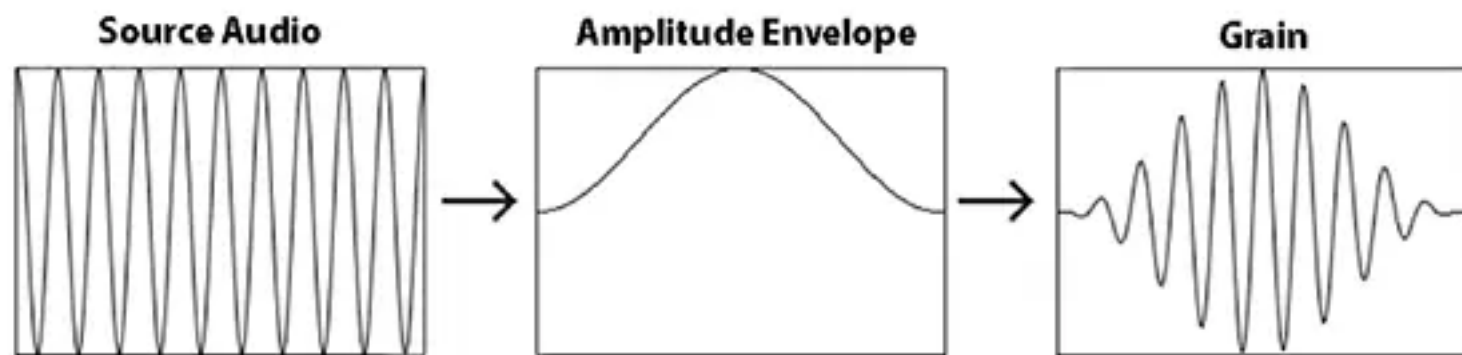


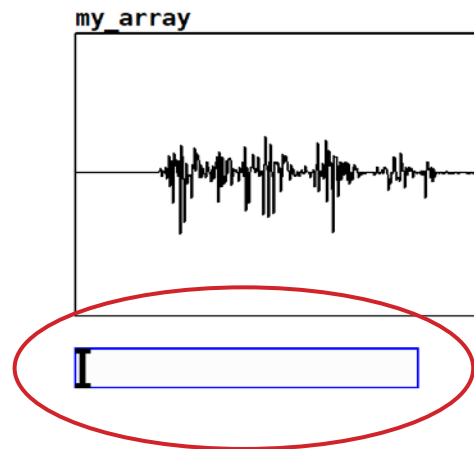
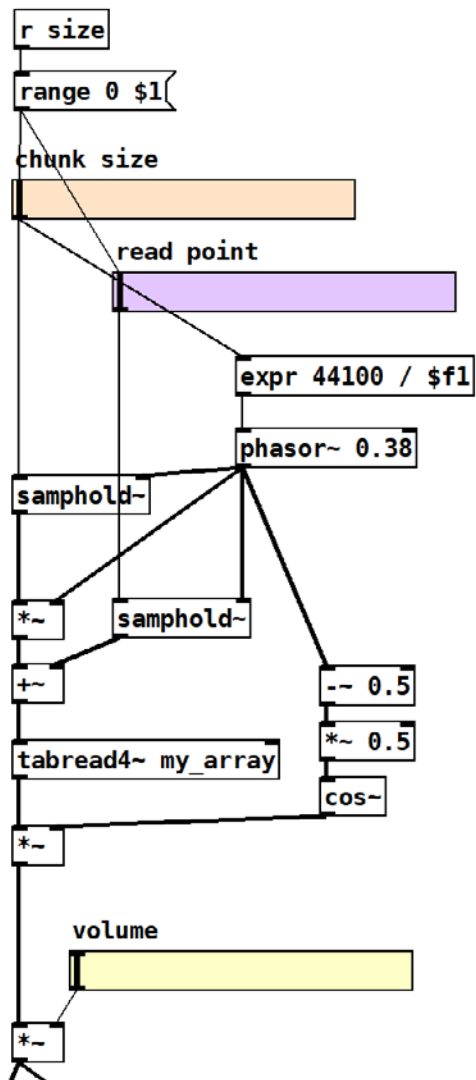
shapes the grain
to avoid clicks

my_array



```
read -resize mysound.wav my_array  
soundfiler  
send size
```

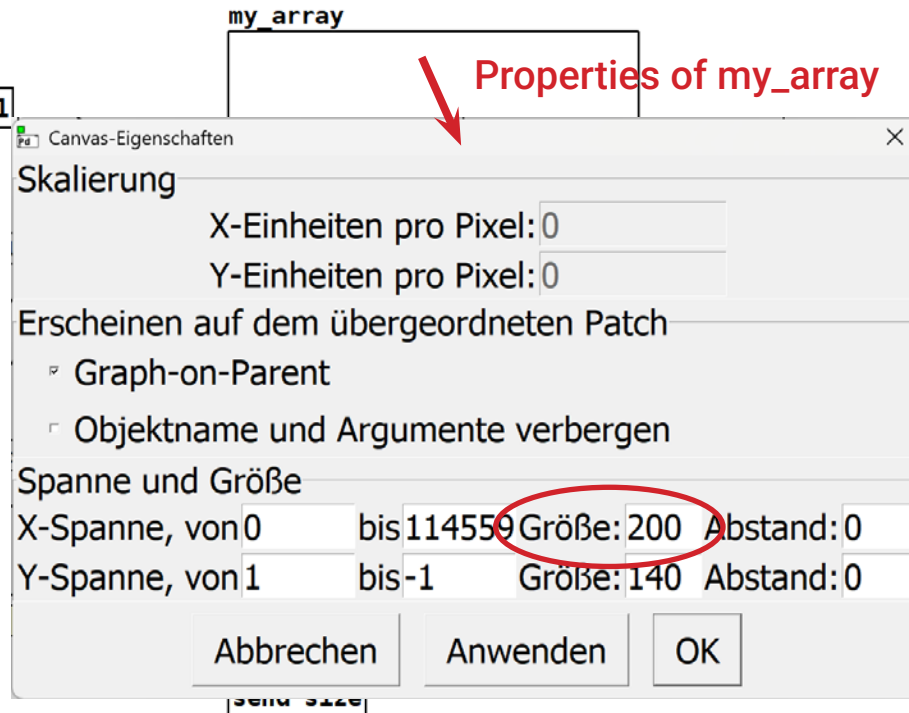
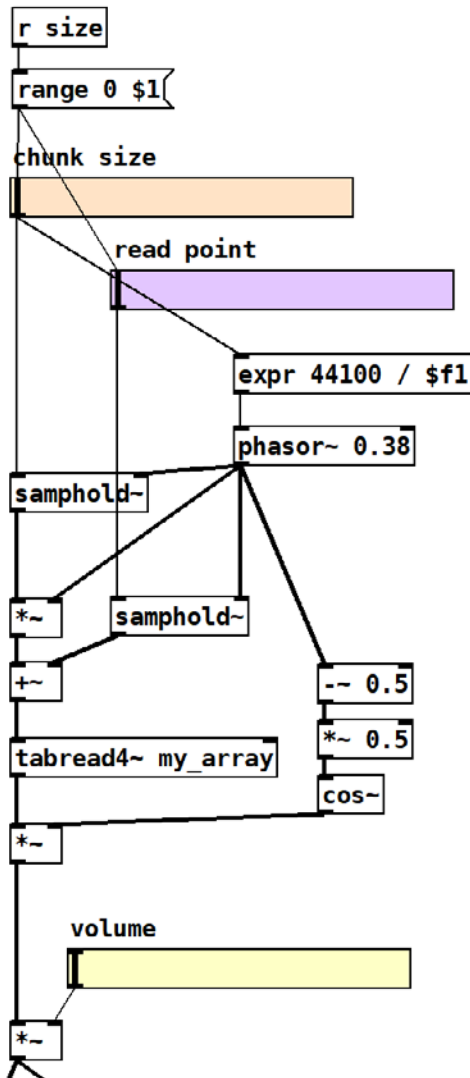


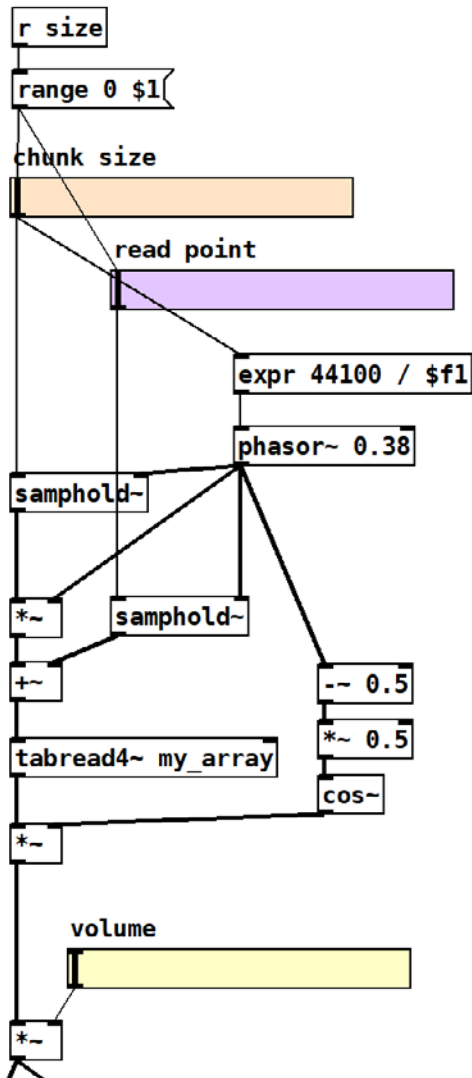


```

read -resize mysound.wav my_array
soundfiler
send size

```





set to same size as array!

my_array

Properties of slider

Slider (horiz.) Eigenschaften

Breite: 200 Höhe: 19

Ausgabebereich
 Unterer: 0 Oberer: 127

Parameter
 linear — Kein Init — Gleichbleibend bei Klick —

Messages
 Sendesymbol:
 Empfangssymbol:

Beschriftung

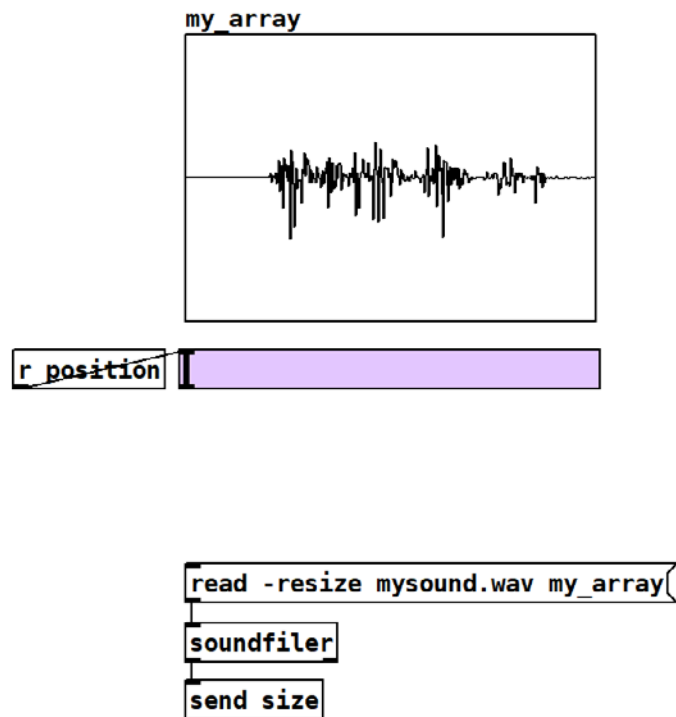
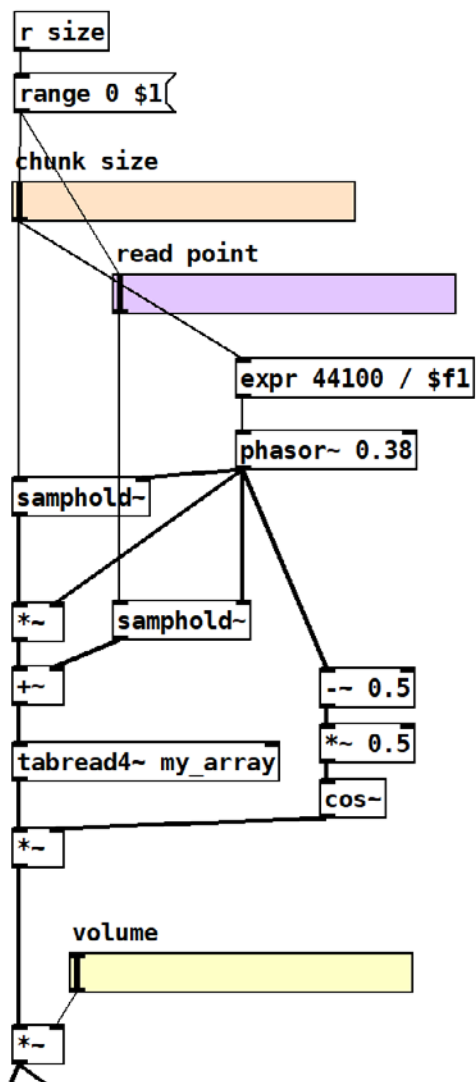
X-Versatz: -2 Y-Versatz: -10

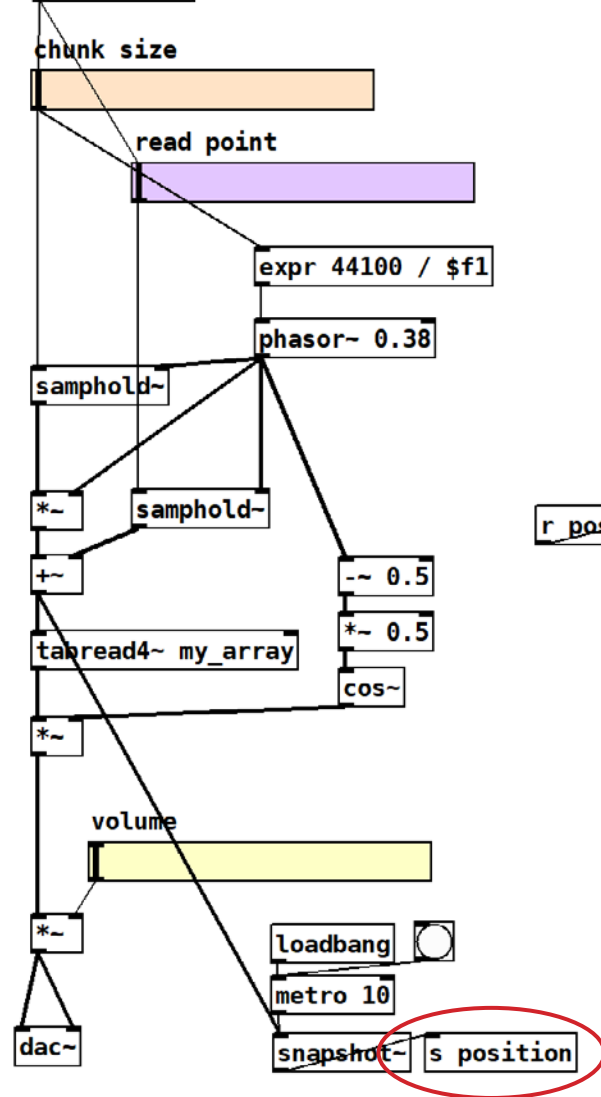
DejaVu Sans Mono Größe: 12

Farben
 * Hintergrund * Vordergrund * Beschriftung

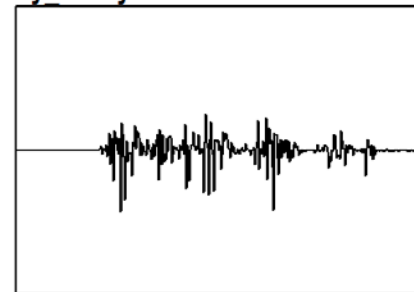
Farbe einstellen o=| |o Test Beschriftung

Abbrechen Anwenden OK

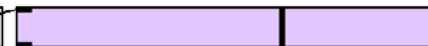




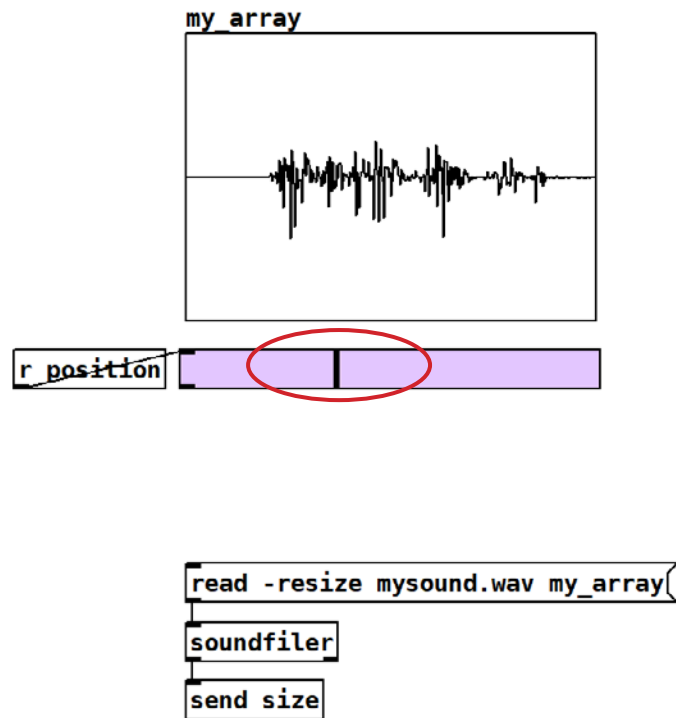
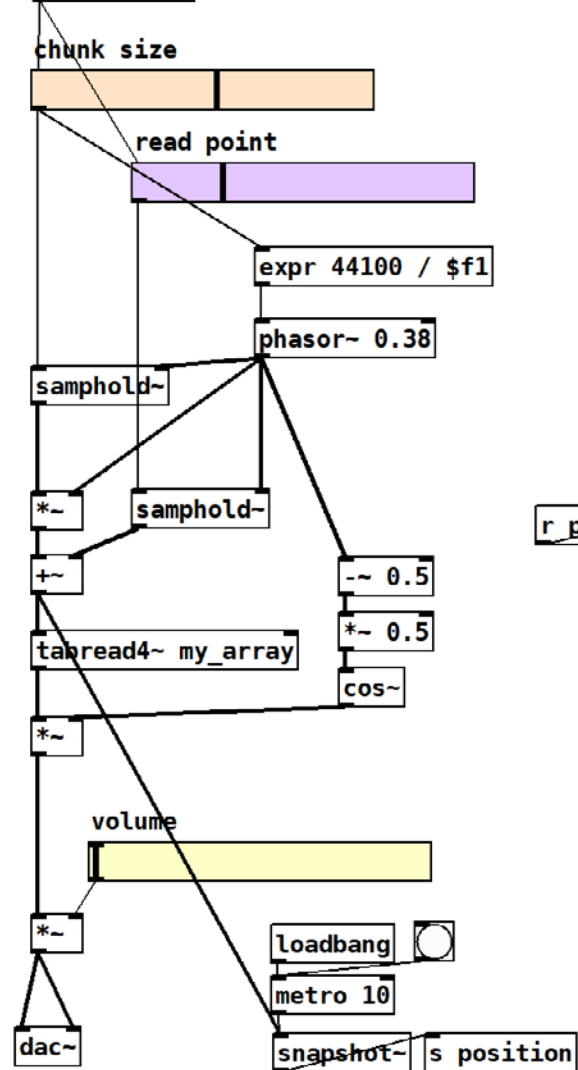
my_array

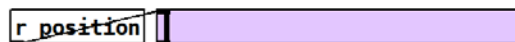
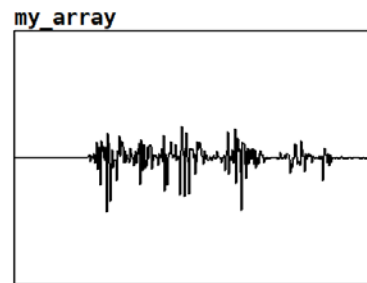
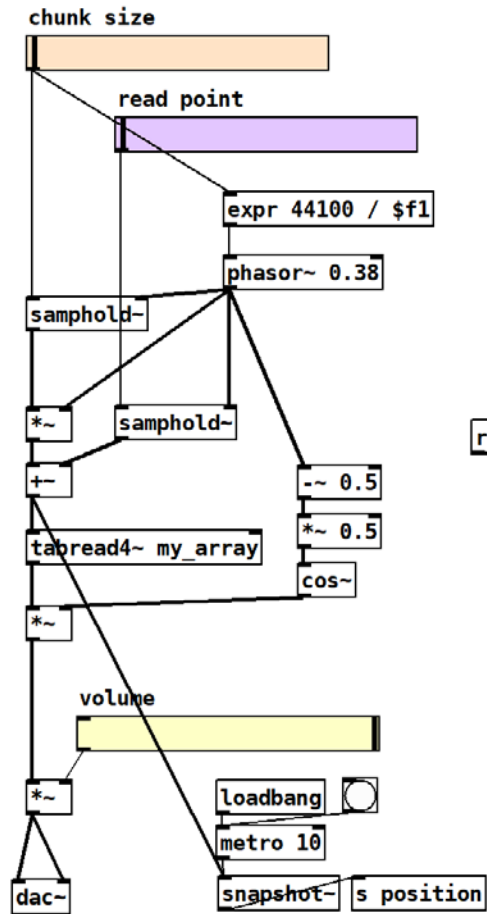


r position



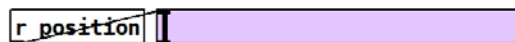
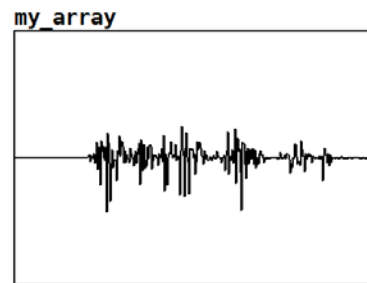
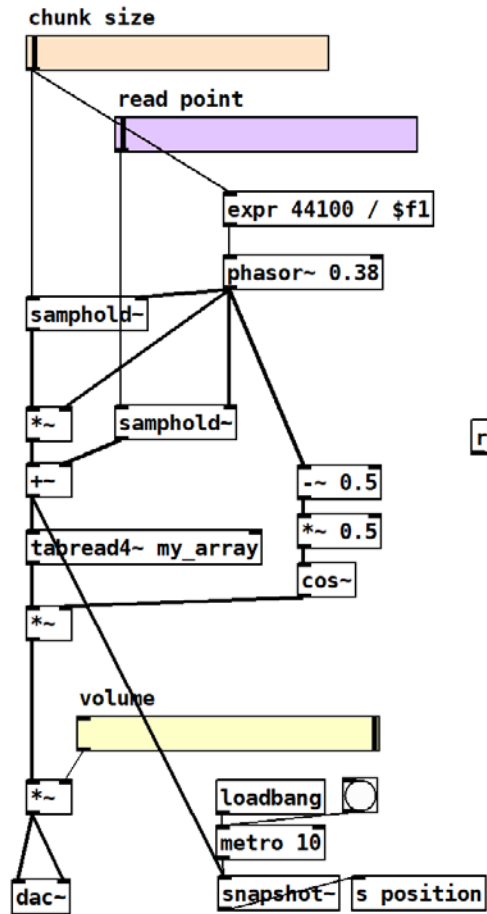
```
read -resize mysound.wav my_array  
soundfiler  
send size
```





```
read -resize mysound.wav my_array  
soundfiler  
send size
```

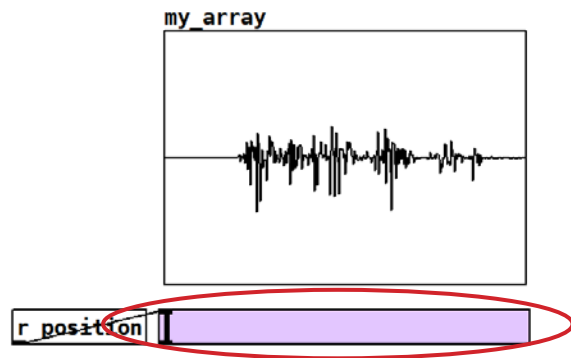
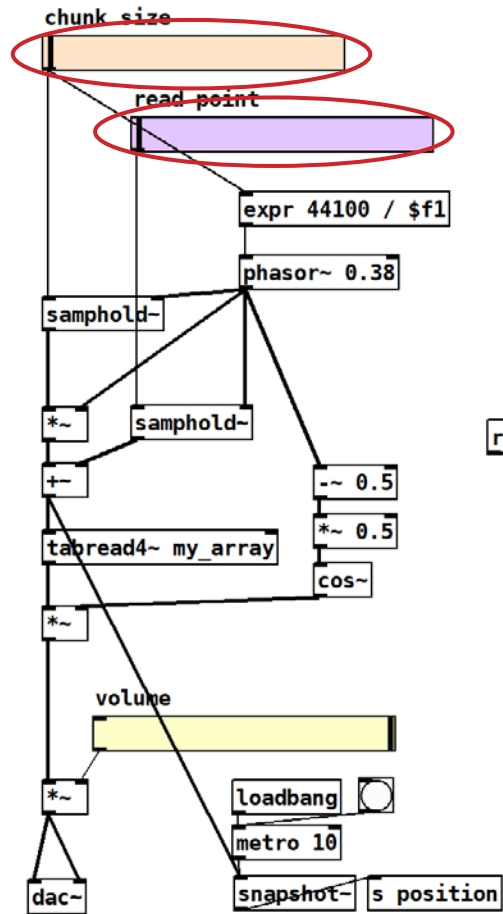
```
r size  
range 0 $f1  
s range
```



```
read -resize mysound.wav my_array
soundfiler
send size
```

```
r size
range 0.1 $1
s range
```

avoid divisions by 0



```
read -resize mysound.wav my_array  
soundfiler  
send size  
  
r size  
range 0.1 $1  
s range
```

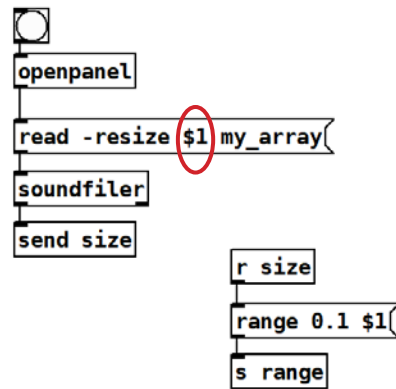
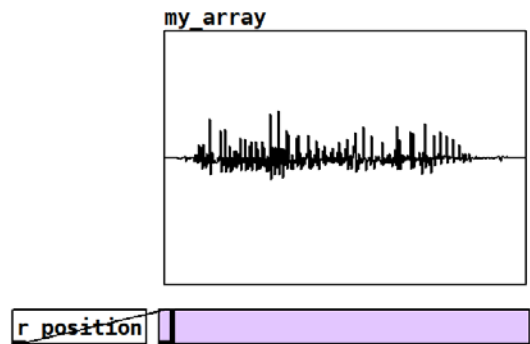
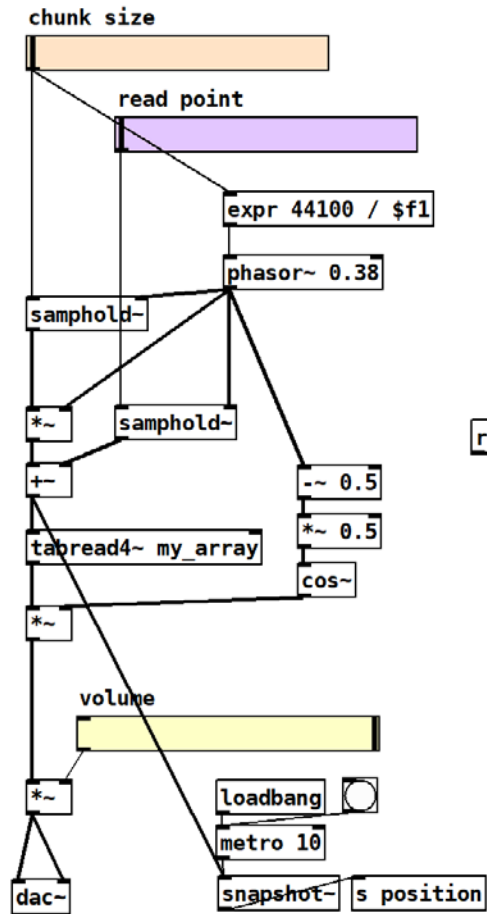
Properties of sliders

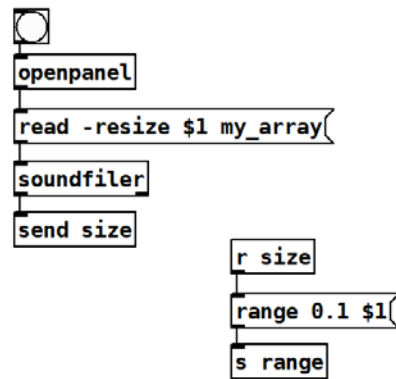
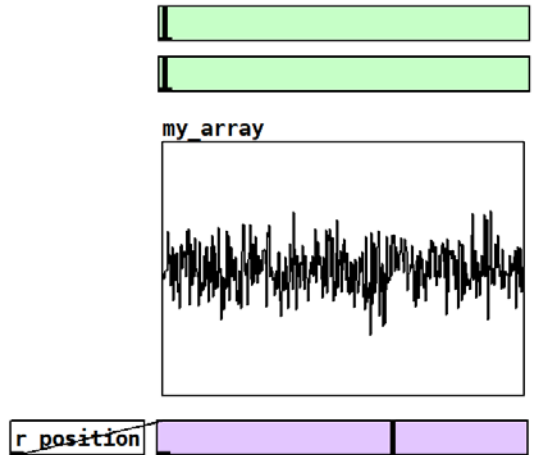
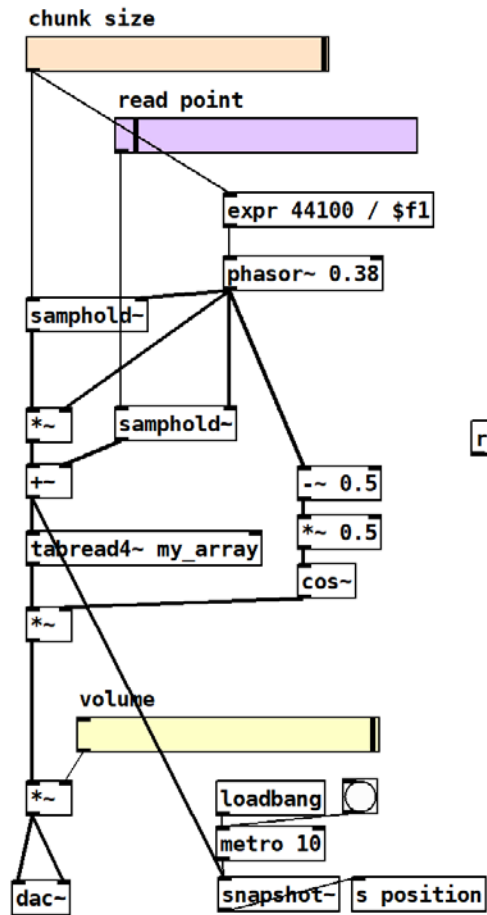
The image shows a Pure Data patch on the left and its corresponding 'Slider (horiz.) Eigenschaften' (Slider (horiz.) Properties) window on the right. A red arrow points from the 'chunk size' slider in the patch to the properties window.

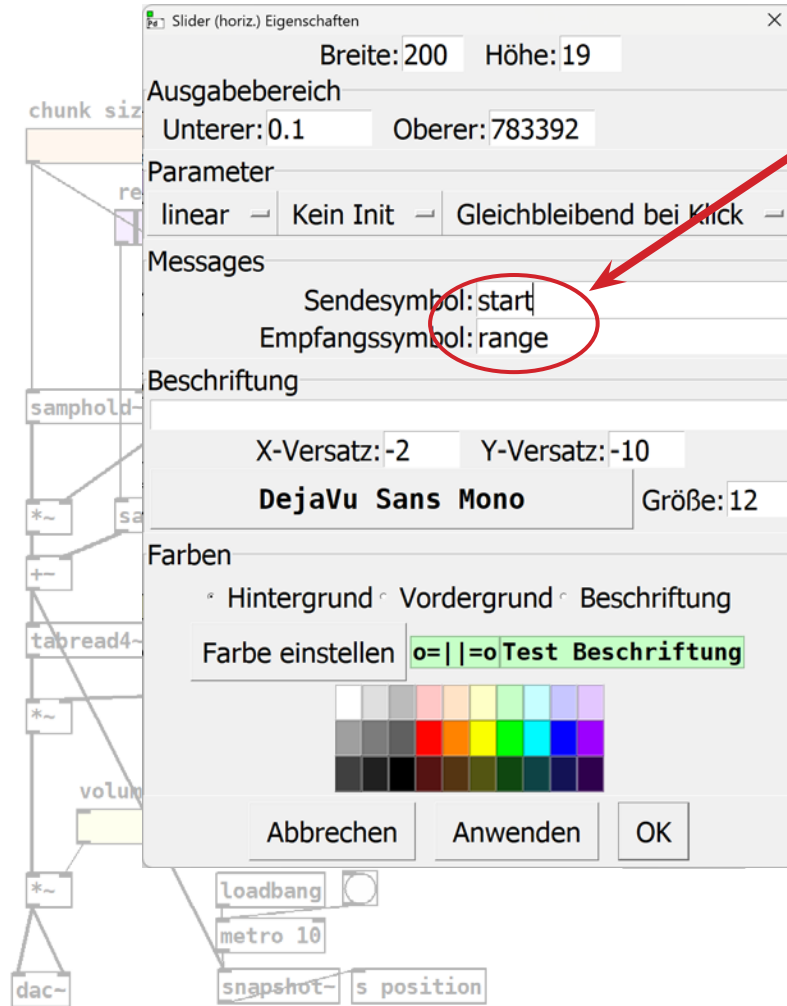
Slider (horiz.) Eigenschaften

- Breite: 162 Höhe: 19
- Ausgabebereich: Unterer: 0 Oberer: 114560
- Parameter: linear ☐ Kein Init ☐ Gleichbleibend bei Klick ☐
- Messages: Sendesymbol: Empfangssymbol: range
- Beschriftung: chunk size X-Versatz: -2 Y-Versatz: -10 DejaVu Sans Mono Größe: 12
- Farben: Hintergrund Vordergrund Beschriftung
Farbe einstellen: Test Beschriftung
- Abbrechen Anwenden OK

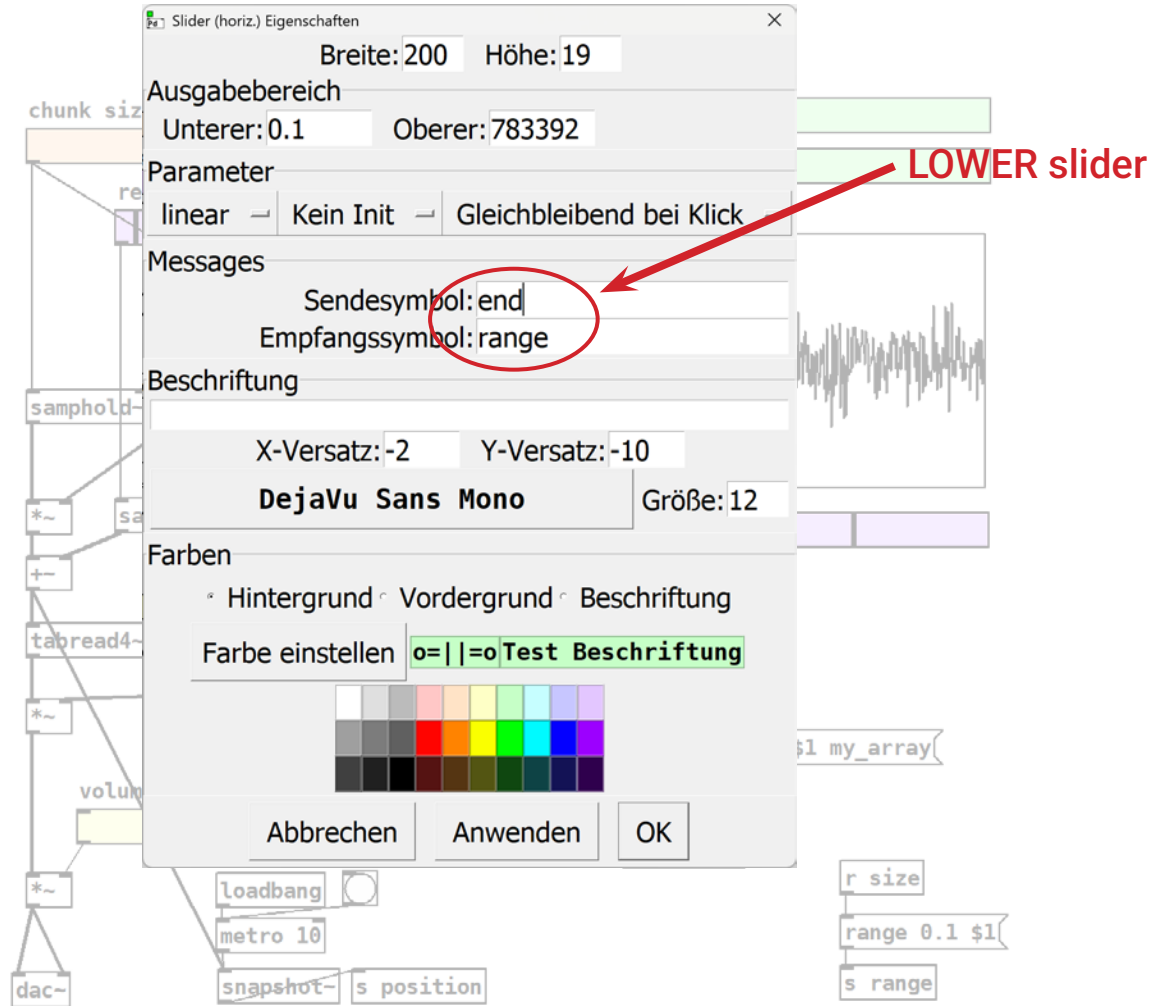
The patch on the left includes objects such as `chunk size`, `read point`, `expr 44`, `phasor~`, `samphold~`, `*~`, `tabread4~ my_array`, `volume`, `loadbang`, `metro 16`, `snapshot~`, `s position`, `s range`, and `dac~`.







UPPER slider



r start

r end

chunk size

read point

expr 44100 / \$f1

phasor~ 0.38

samphold~

*~

samphold~

+~

~ 0.5

tabread4~ my_array

*~ 0.5

cos~

*~

volume

*~

loadbang

metro 10

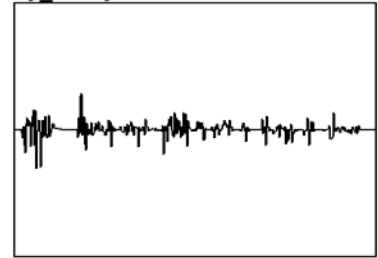
snapshot~

s position

dac~



my_array



r_position



openpanel

read -resize \$1 my_array

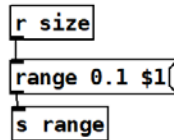
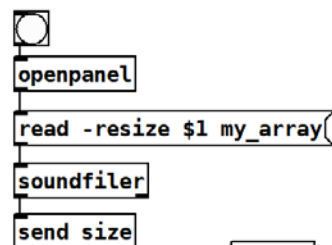
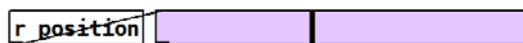
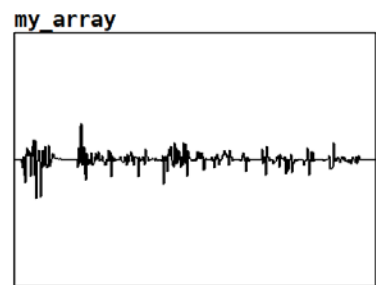
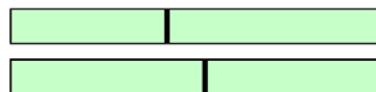
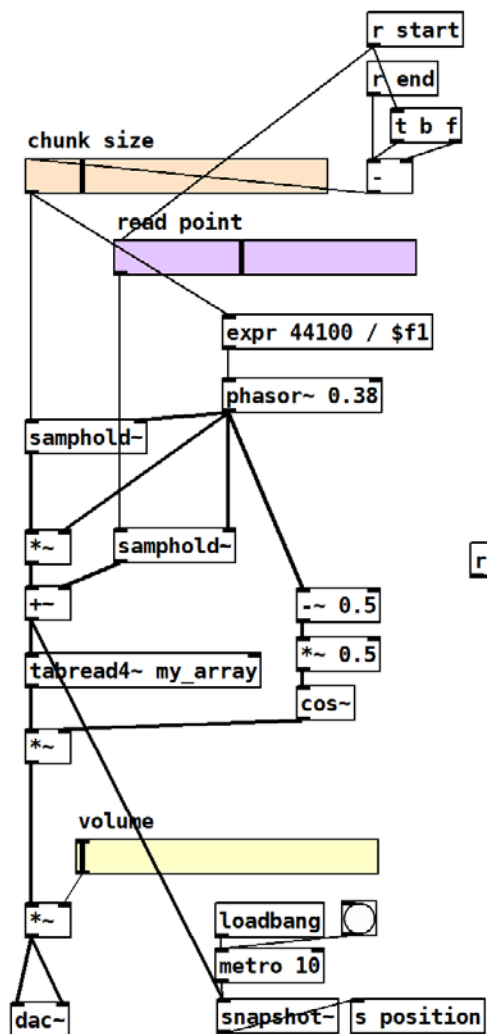
soundfiler

send size

r size

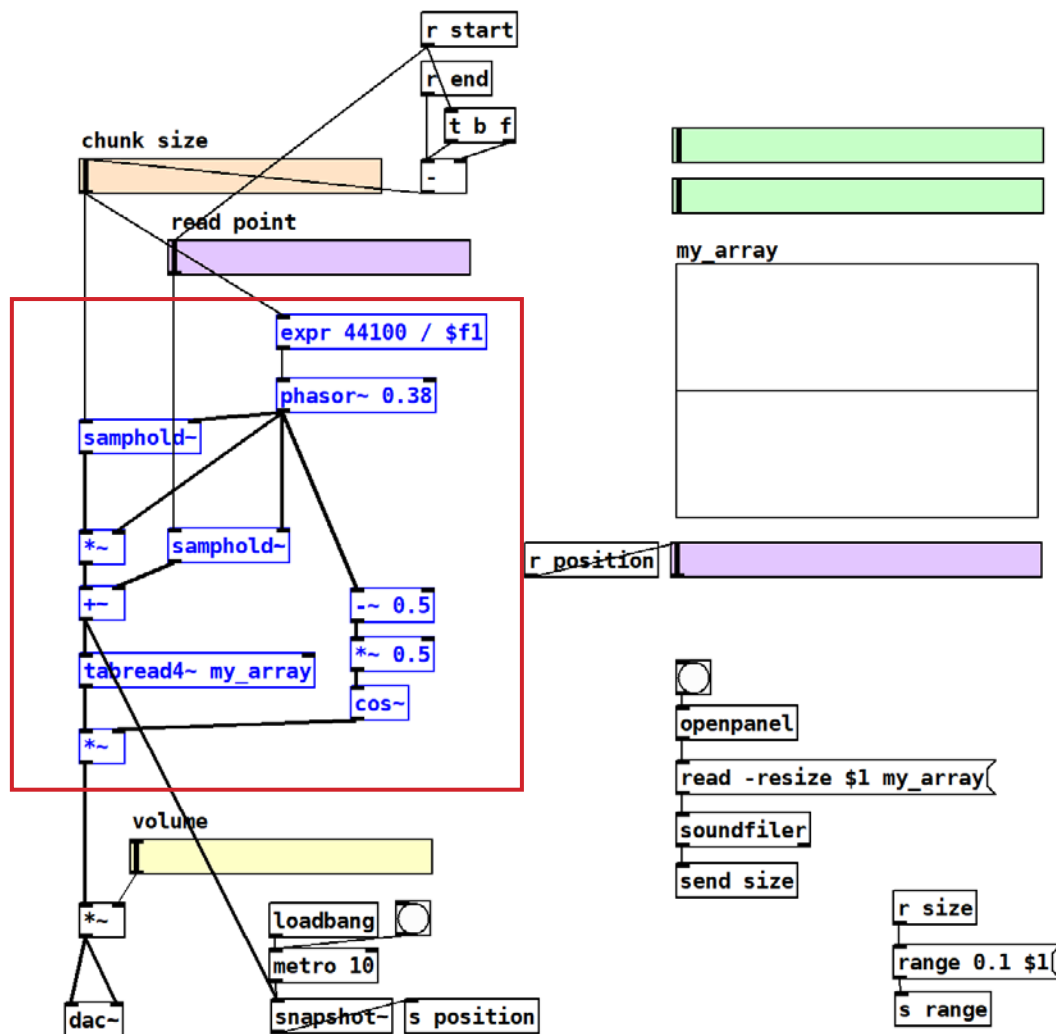
range 0.1 \$1

s range

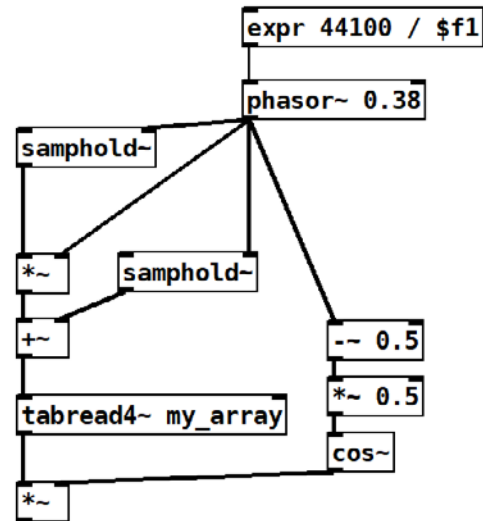


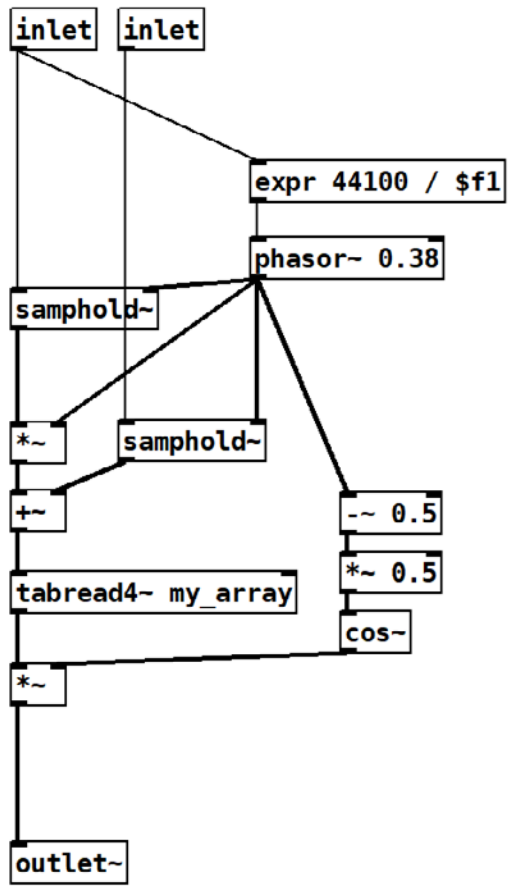
MULTIPLE GRAINS

select & cut

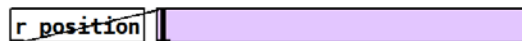
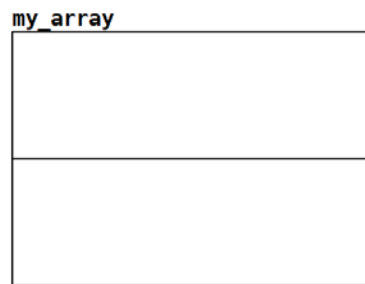
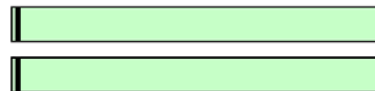
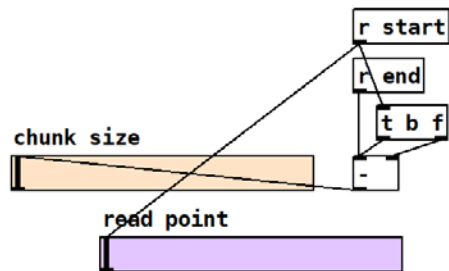


paste in new window



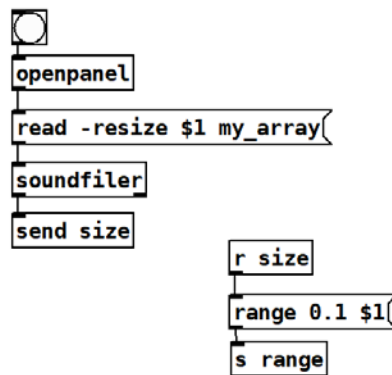
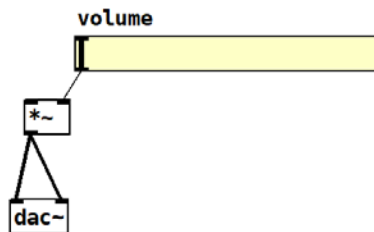


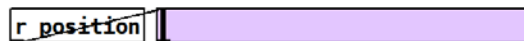
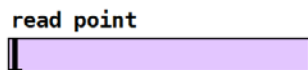
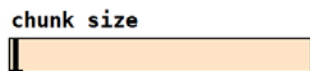
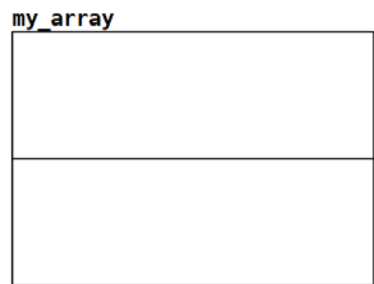
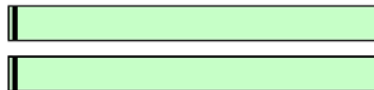
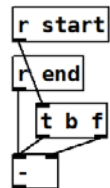
save as
TAM_grain



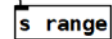
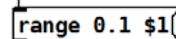
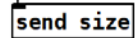
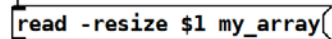
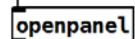
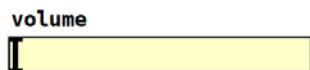
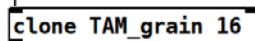
12 overlapping grains

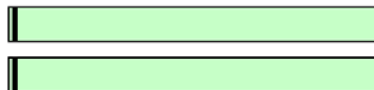
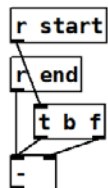
clone TAM_grain 16



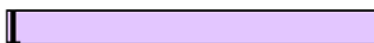
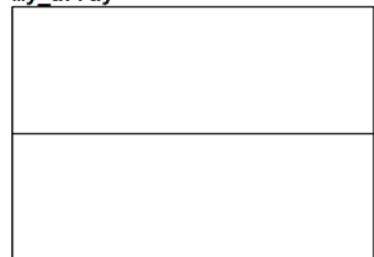


next \$1 send value to next instance





my_array



openpanel

read -resize \$1 my_array

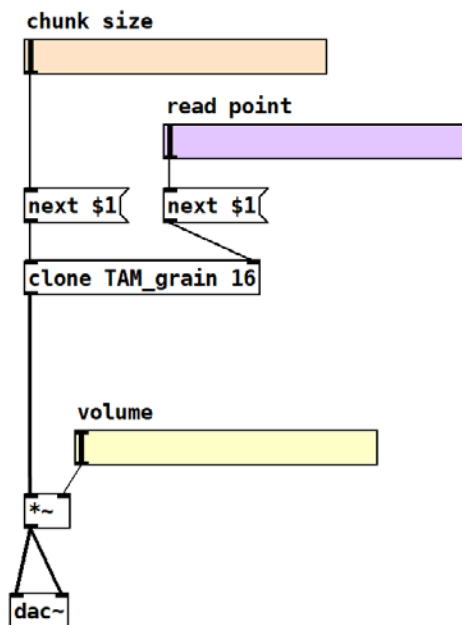
soundfiler

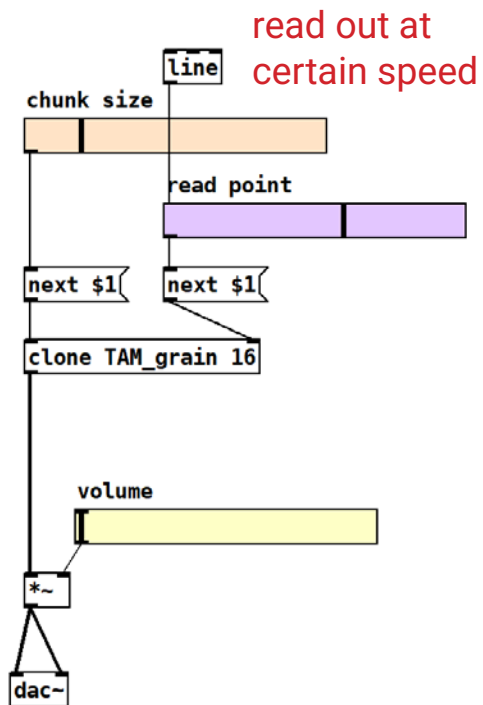
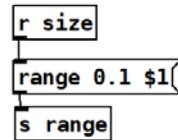
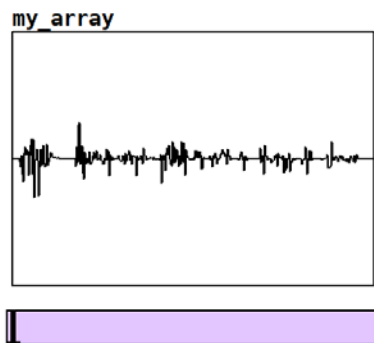
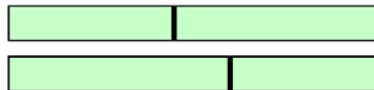
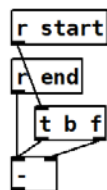
send size

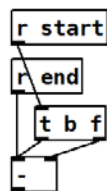
r size

range 0.1 \$1

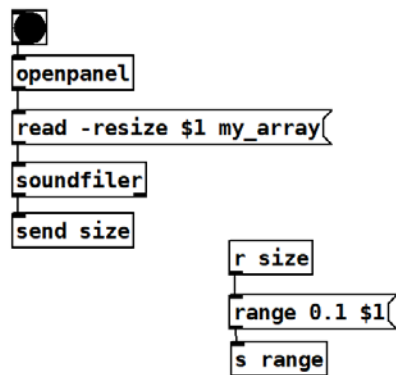
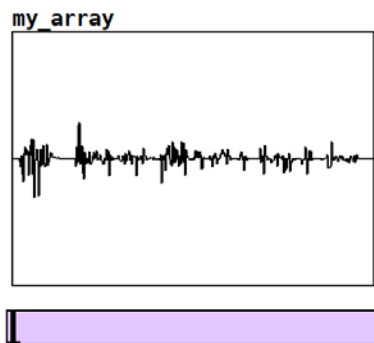
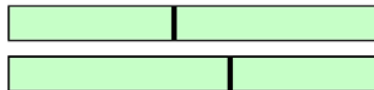
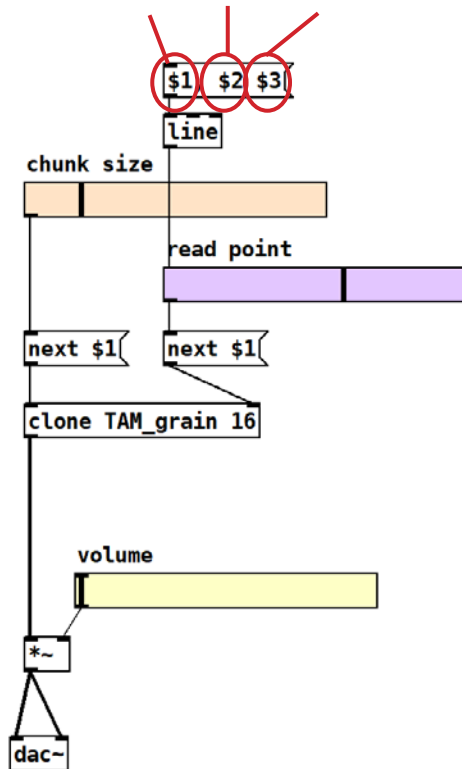
s range

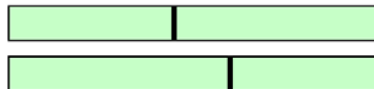
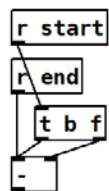




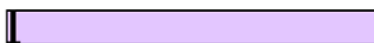
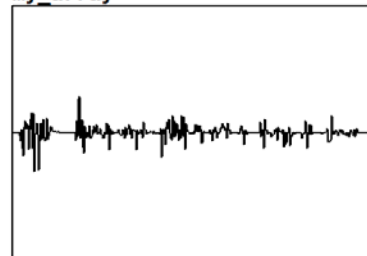


start end duration





my_array



openpanel

read -resize \$1 my_array

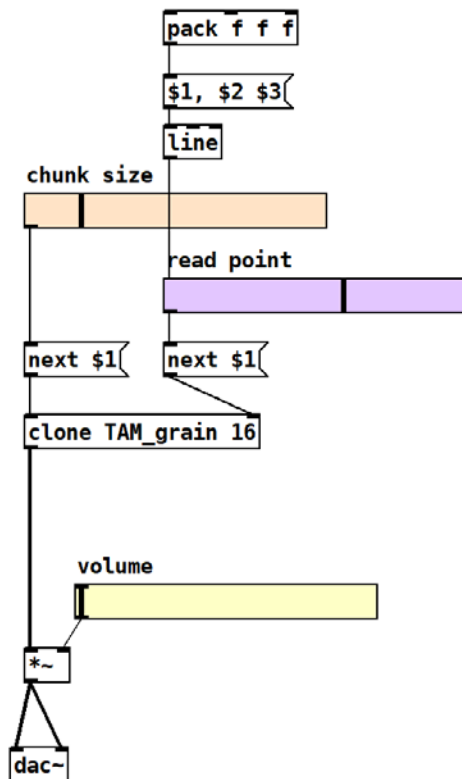
soundfiler

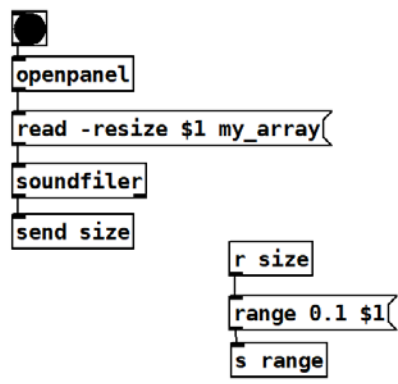
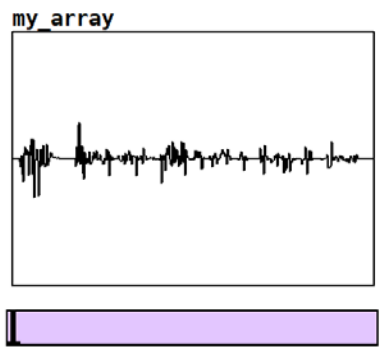
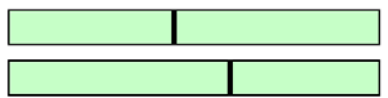
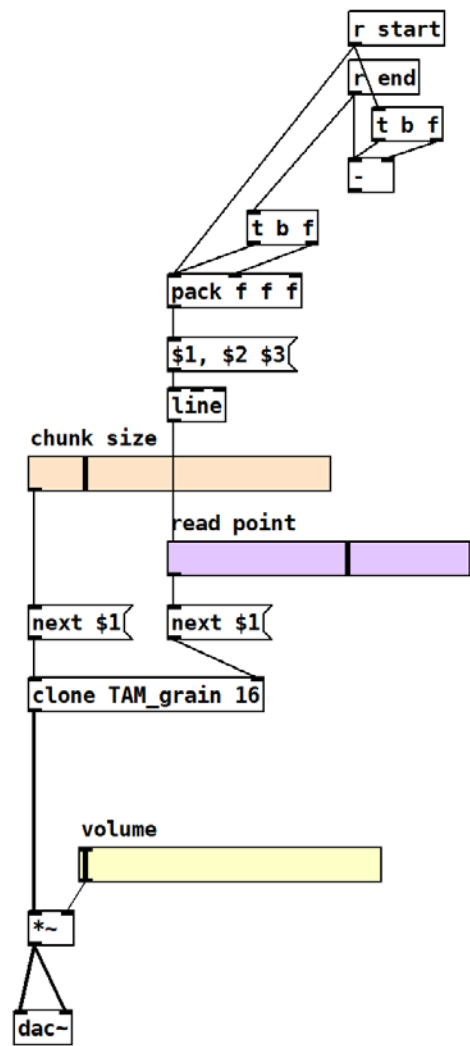
send size

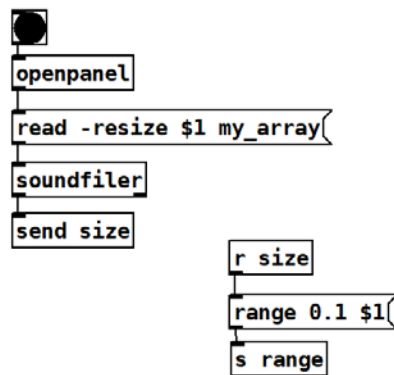
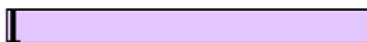
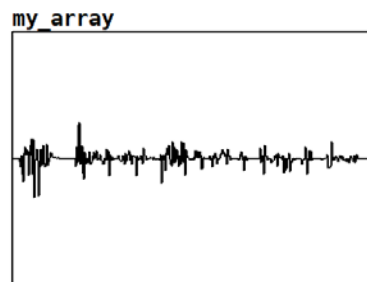
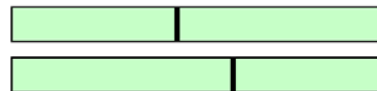
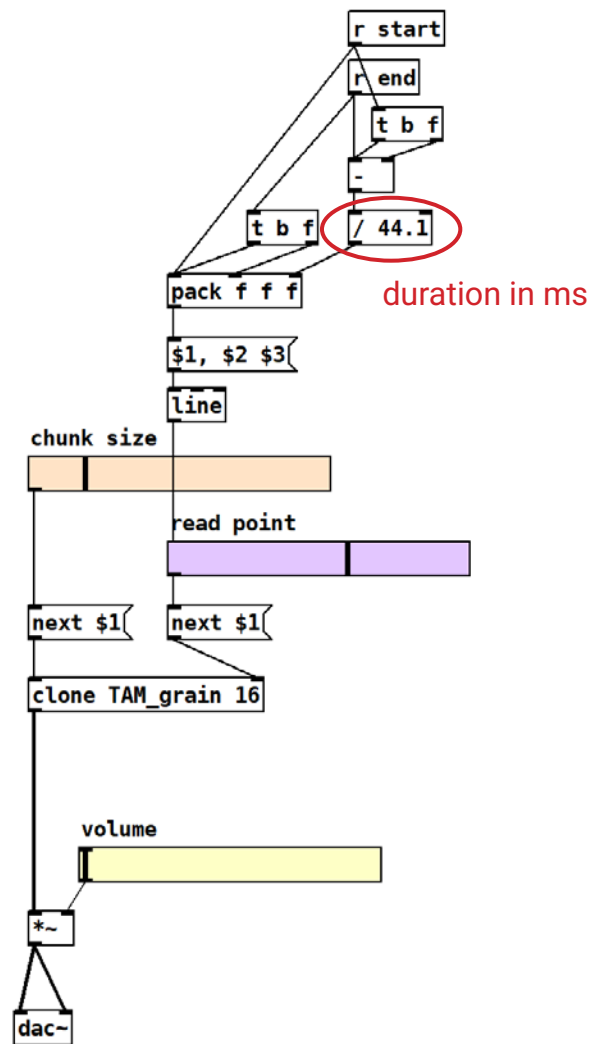
r size

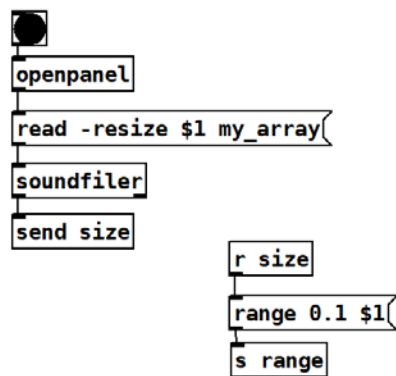
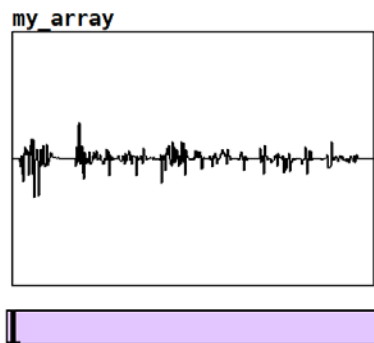
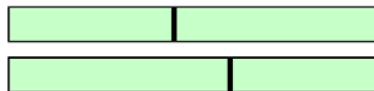
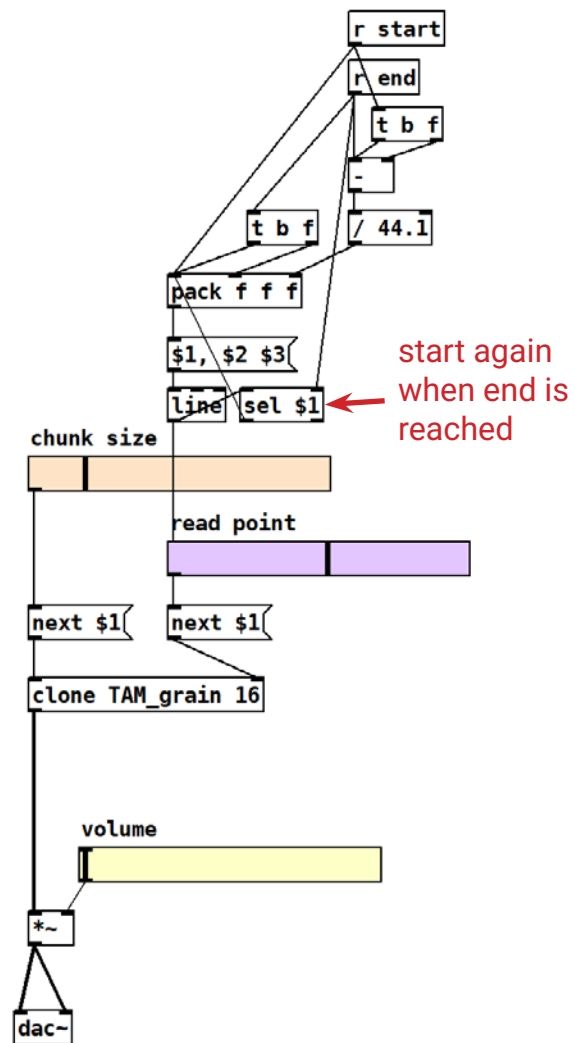
range 0.1 \$1

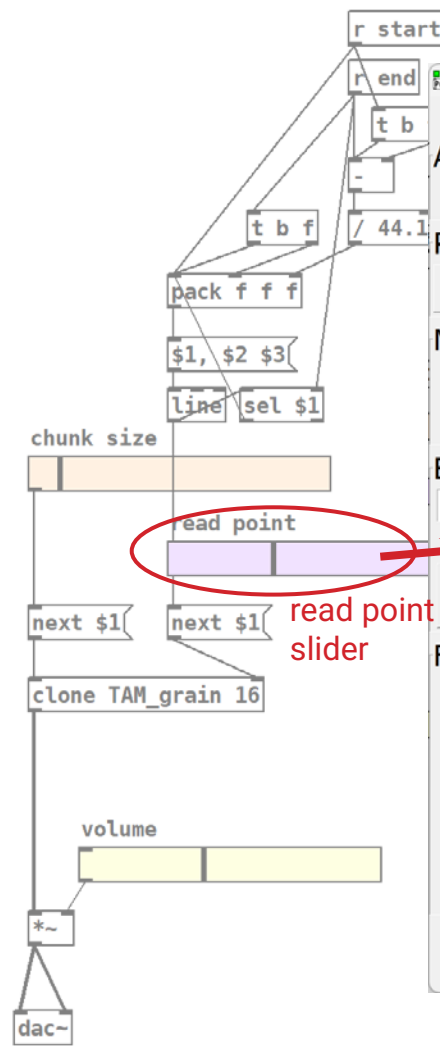
s range











Slider (horiz.) Eigenschaften

Breite: 162 Höhe: 19

Ausgabebereich
Unterer: 0.1 Oberer: 149499

Parameter
linear — Kein Init — Gleichbleibend bei Klick —

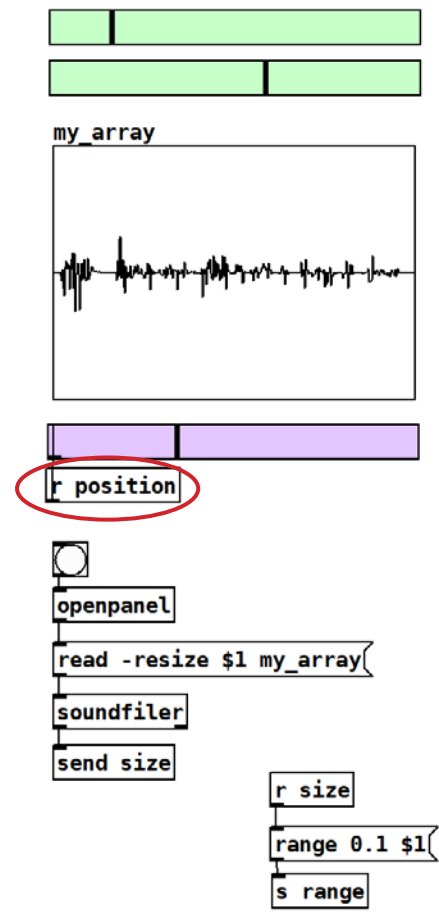
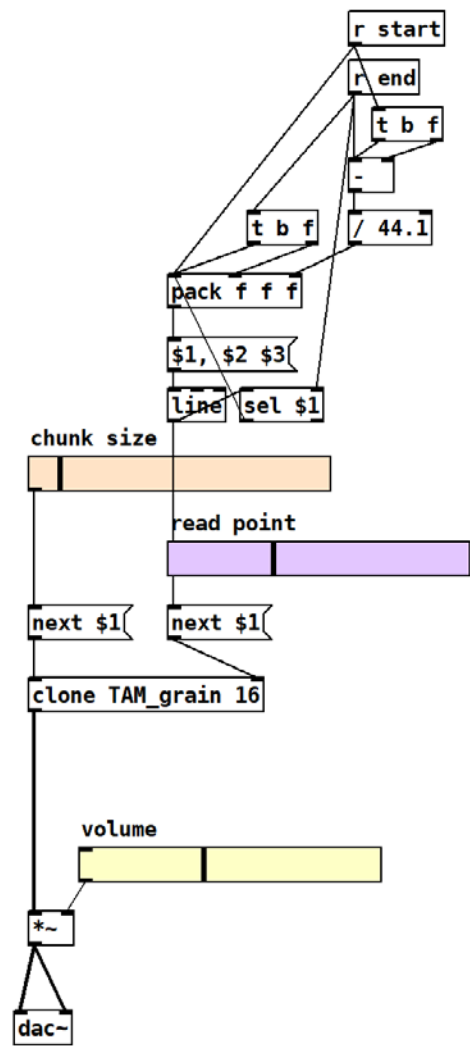
Messages
Sendesymbol: position
Empfangssymbol: range

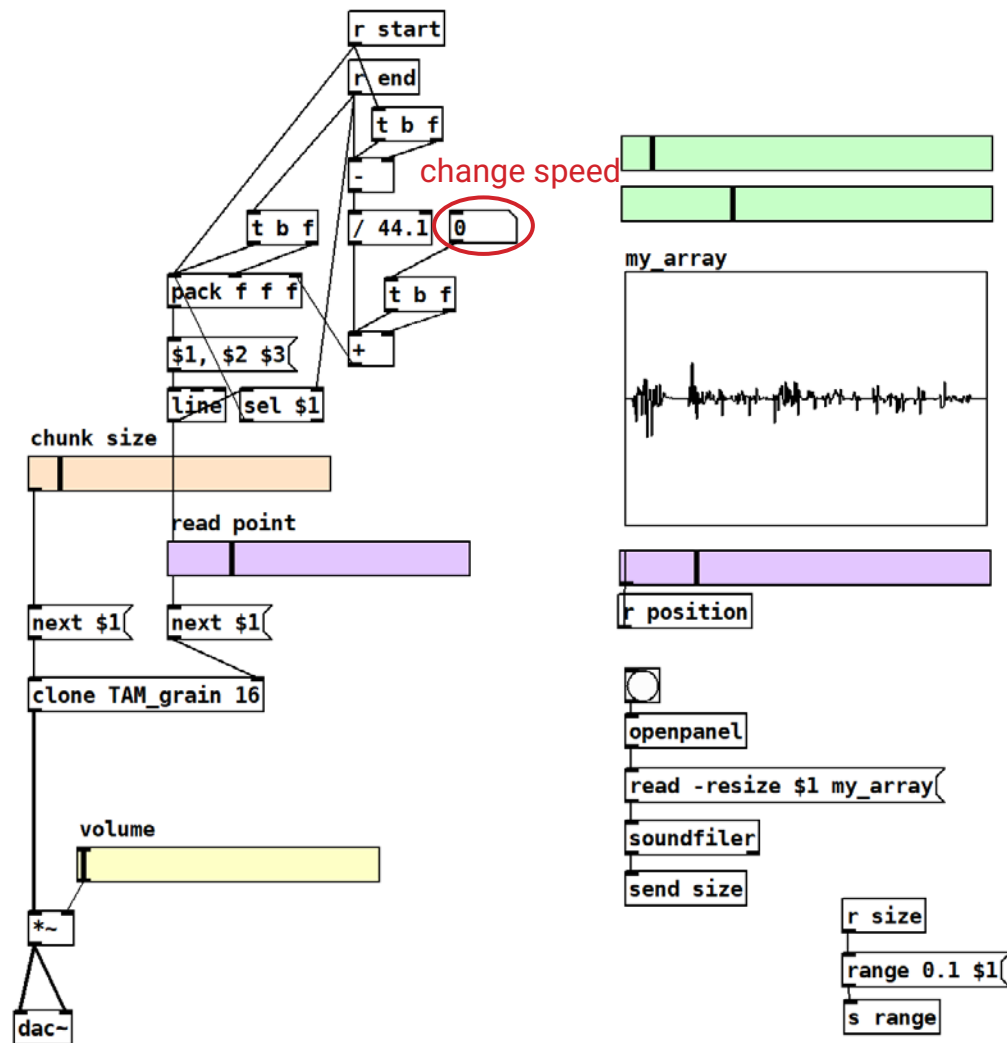
Beschriftung
read point
X-Versatz: -2 Y-Versatz: -10
DejaVu Sans Mono Größe: 12

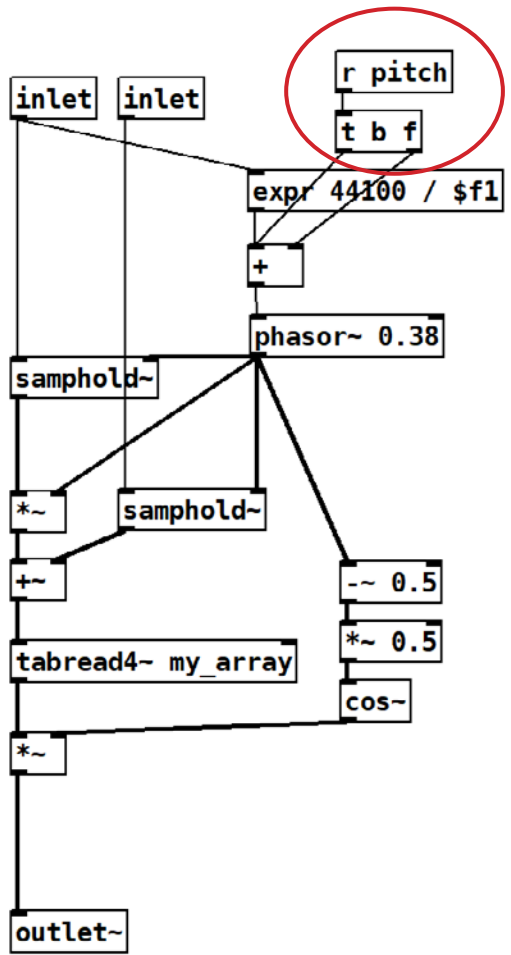
Farben
Hintergrund Vordergrund Beschriftung
Farbe einstellen o=||=o Test Beschriftung

Abbrechen Anwenden OK

s range

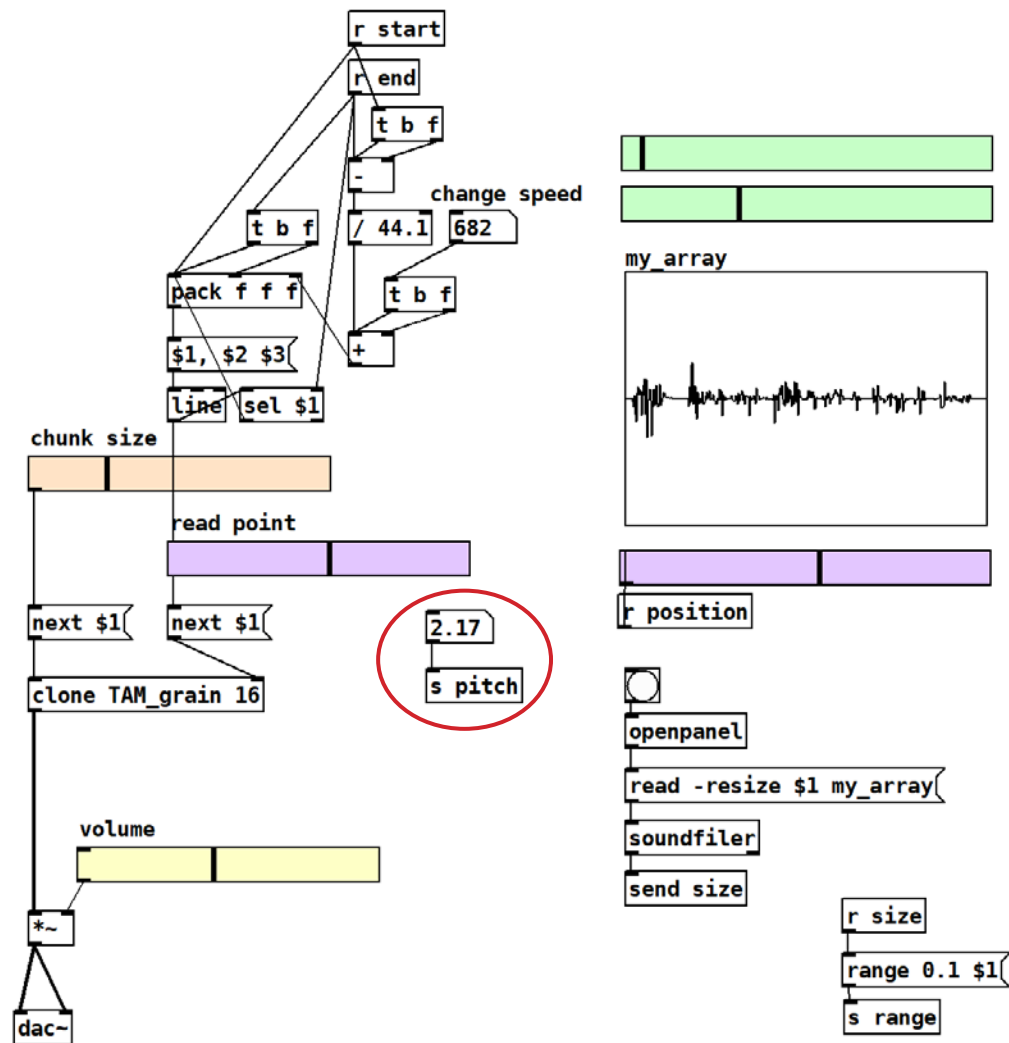






change pitch

TAM_grain



WAVETABLE SYNTHESIS

MORPHING BETWEEN WAVETABLES

Objekt	Ctrl+1
Message	Ctrl+2
Nummer	Ctrl+3
Liste	Ctrl+4
Symbol	
Kommentar	Ctrl+5
Taster (Bang)	Shift+Ctrl+B
Schalter	Shift+Ctrl+T
Number2	Shift+Ctrl+N
Slider (vert.)	Shift+Ctrl+V
Slider (horiz.)	Shift+Ctrl+J
Radio (vert.)	Shift+Ctrl+D
Radio (horiz.)	Shift+Ctrl+I
VU-Meter	Shift+Ctrl+U
Canvas	Shift+Ctrl+C
Graph	Shift+Ctrl+G
Array	Shift+Ctrl+A



Array-Eigenschaften

Array

Name: waveA

Größe: 512

☒ Inhalt speichern

Zeichne als:

- ☐ Polygon
- ☐ Punkte
- ☐ Bézier-Kurve

Array einfügen in:

- ☐ neuen Graph
- ☐ letzten Graph

Abbrechen OK

waveA



waveB



```
;
waveA sinesum 512 1;
waveB sinesum 512 0.5 0.33 0.25
```

creates sine wave
with partials

1. partial

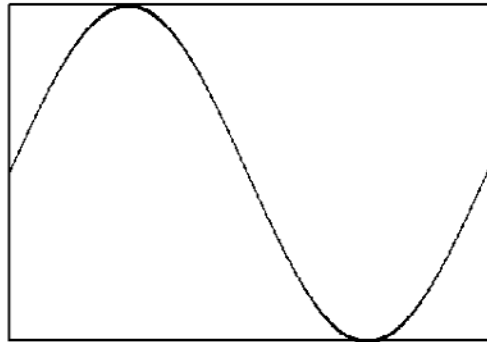
2. partial

3. partial

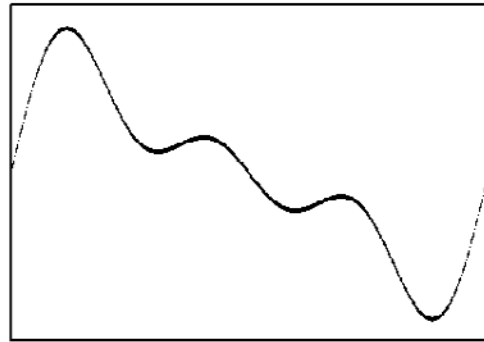
```
;
waveA sinesum 512 1;
waveB sinesum 512 0.5 0.33 0.25
```

click!

waveA



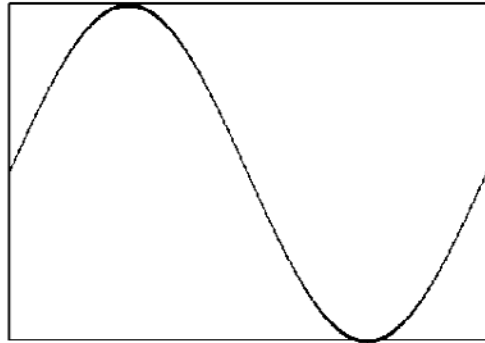
waveB



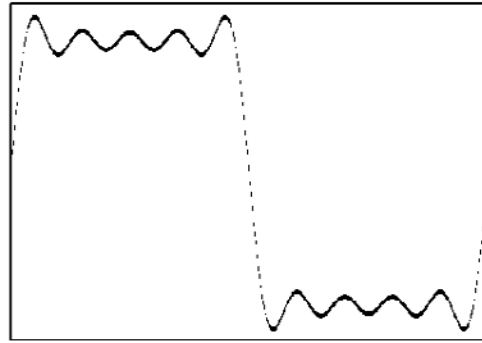
```
;
waveA sinesum 512 1;
waveB sinesum 512 1 0 0.3333 0 0.2 0 0.1429 0 0.1111
```

creates squarewave
(approximate)

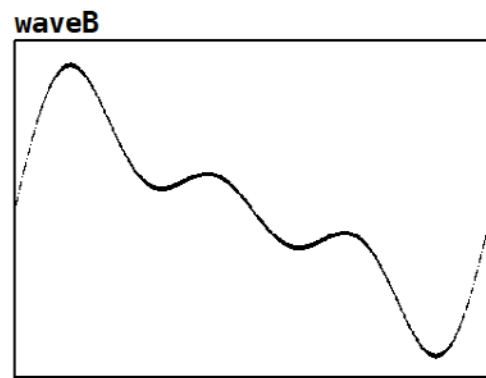
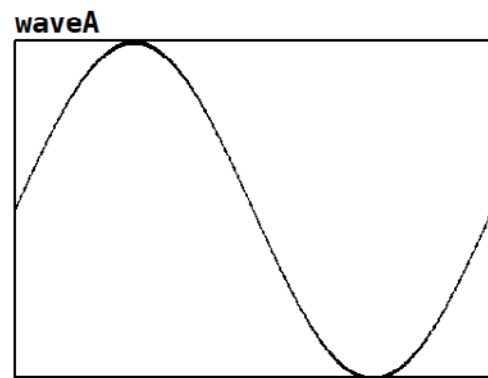
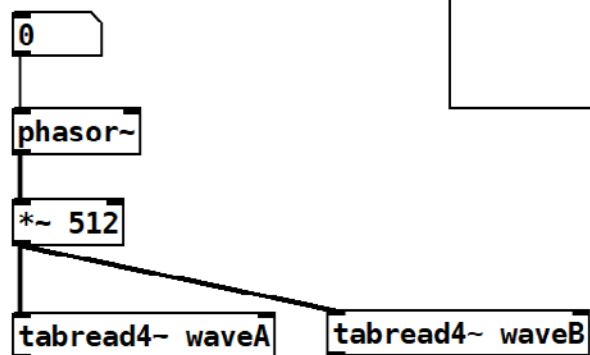
waveA



waveB

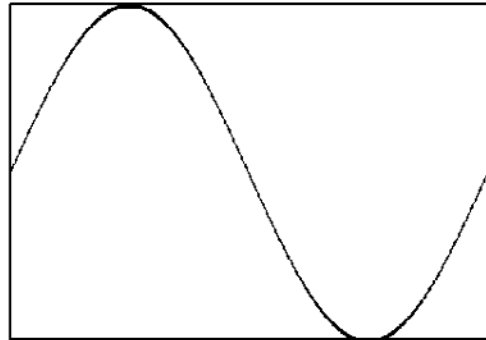


```
;
waveA sinesum 512 1;
waveB sinesum 512 0.5 0.33 0.25
```

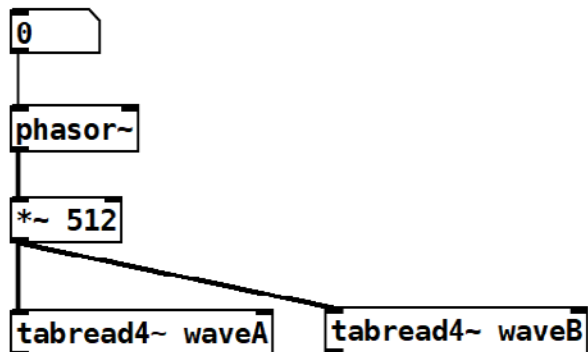
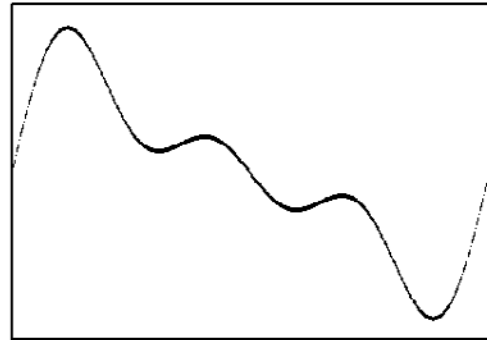


```
;
waveA sinesum 512 1;
waveB sinesum 512 0.5 0.33 0.25
```

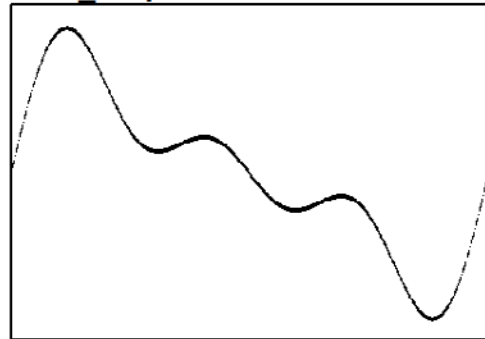
waveA



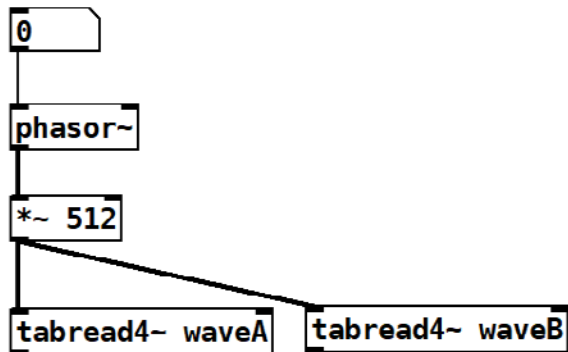
waveB



wave_morph



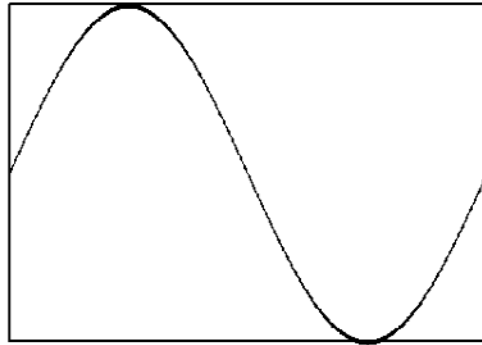
```
;
waveA sinesum 512 1;
waveB sinesum 512 0.5 0.33 0.25
```



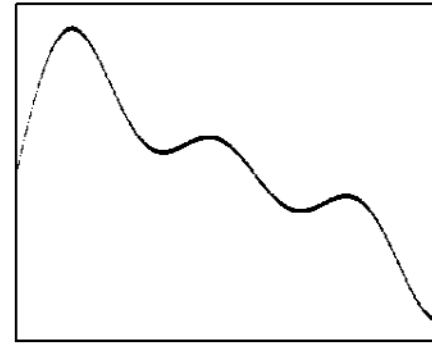
$$\text{fexpr~ } (1 - \$f3) * \$x1[0] + \$f3 * \$x2[0]$$

interpolation between
waveA and waveB

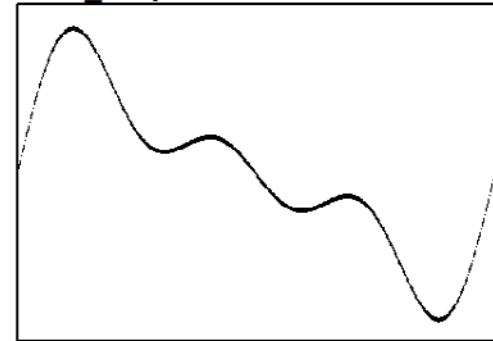
waveA



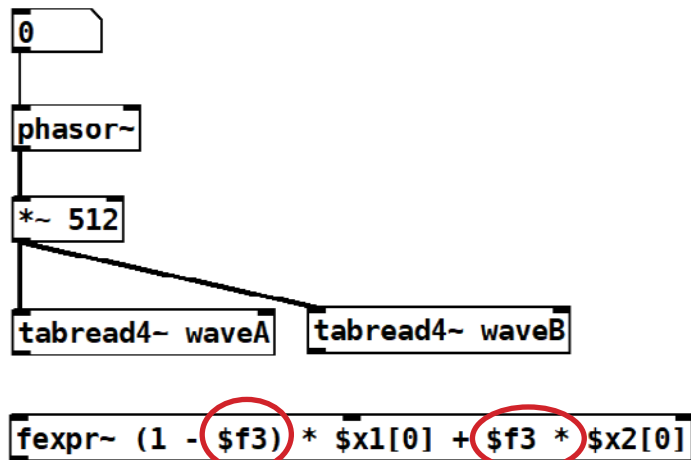
waveB



wave_morph

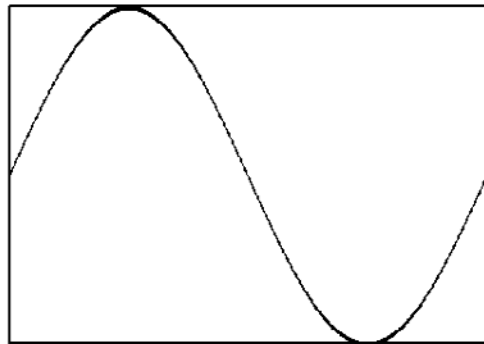


```
;
waveA sinesum 512 1;
waveB sinesum 512 0.5 0.33 0.25
```

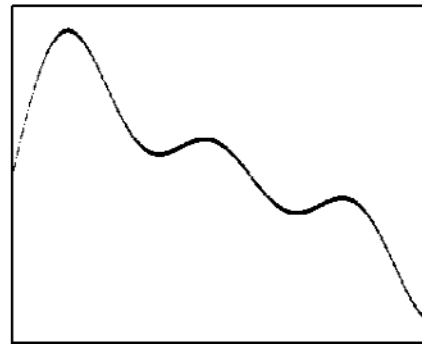


$\$f3$ = morph amount

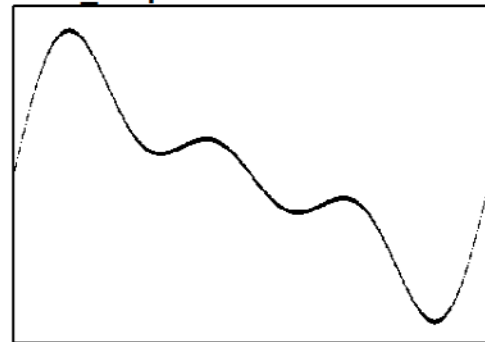
waveA



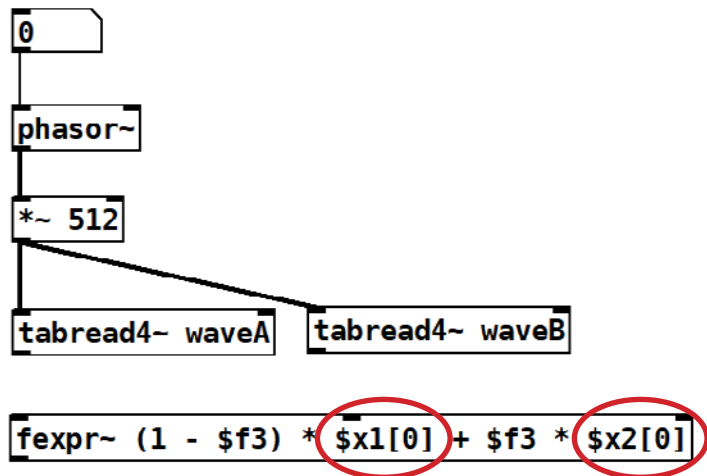
waveB



wave_morph



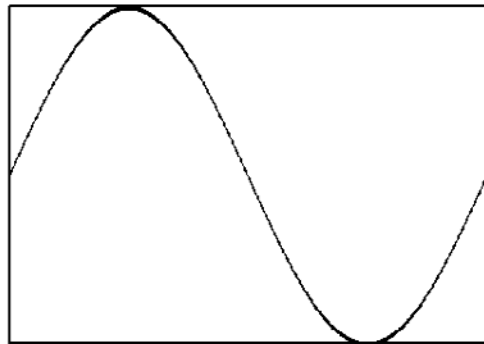
```
;
waveA sinesum 512 1;
waveB sinesum 512 0.5 0.33 0.25
```



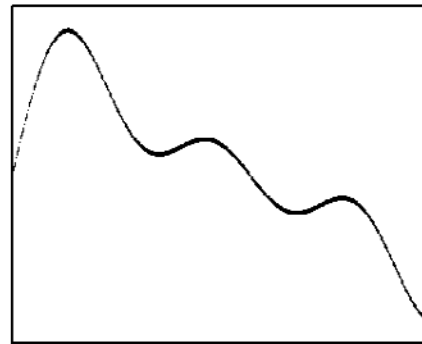
current sample
from waveA

current sample
from waveB

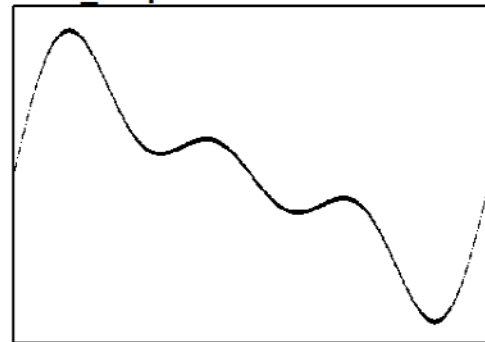
waveA



waveB

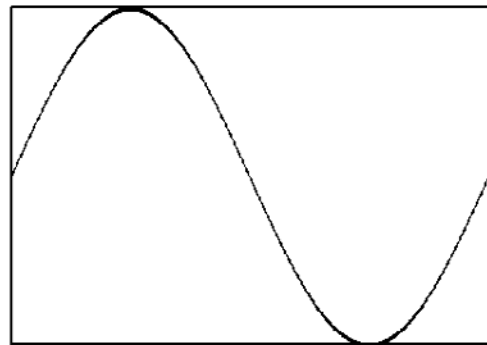


wave_morph

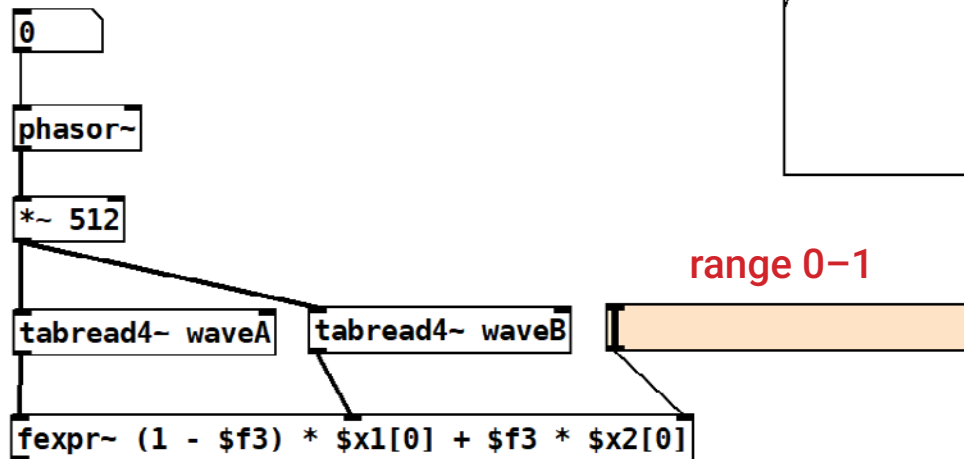
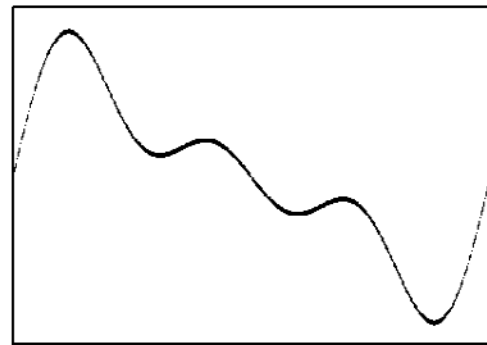


```
;
waveA sinesum 512 1;
waveB sinesum 512 0.5 0.33 0.25
```

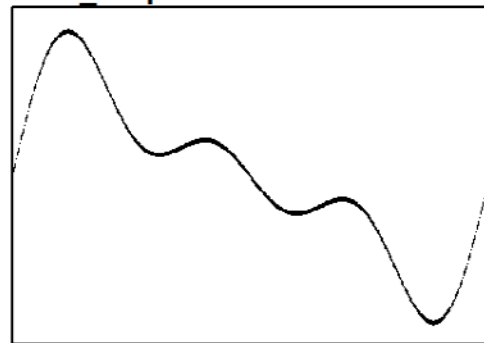
waveA



waveB

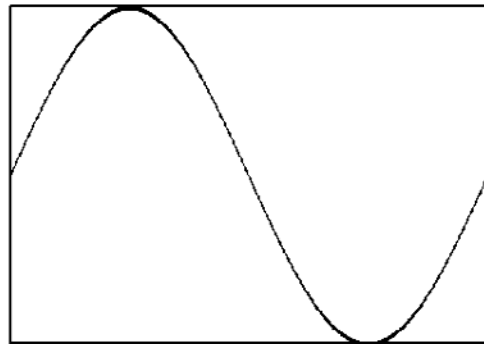


wave_morph

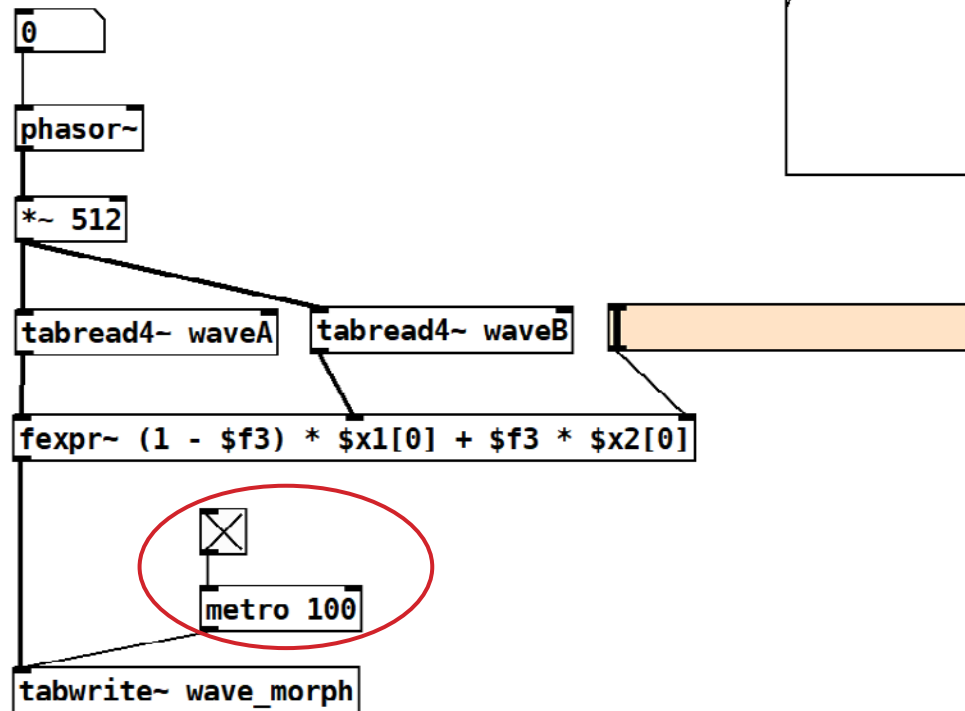
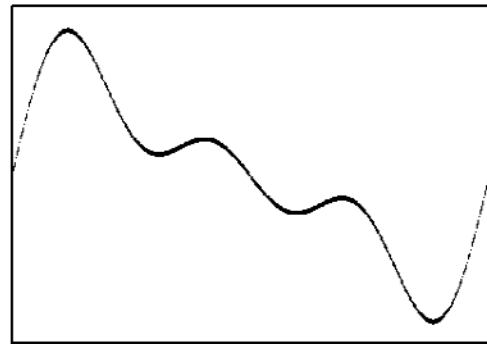


```
;
waveA sinesum 512 1;
waveB sinesum 512 0.5 0.33 0.25
```

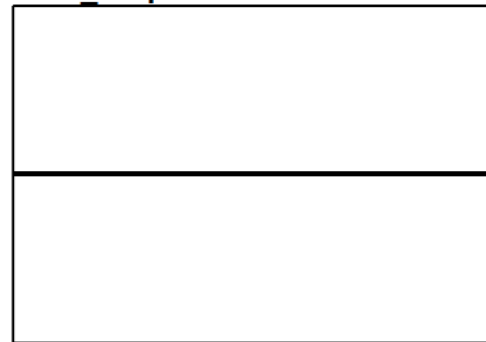
waveA



waveB

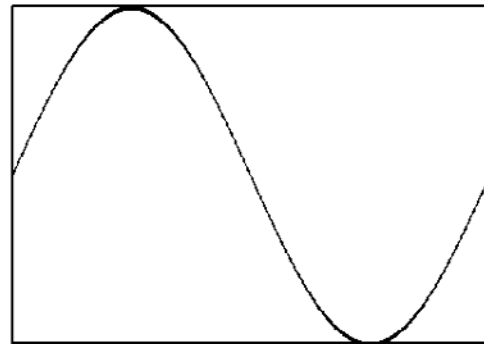


wave_morph

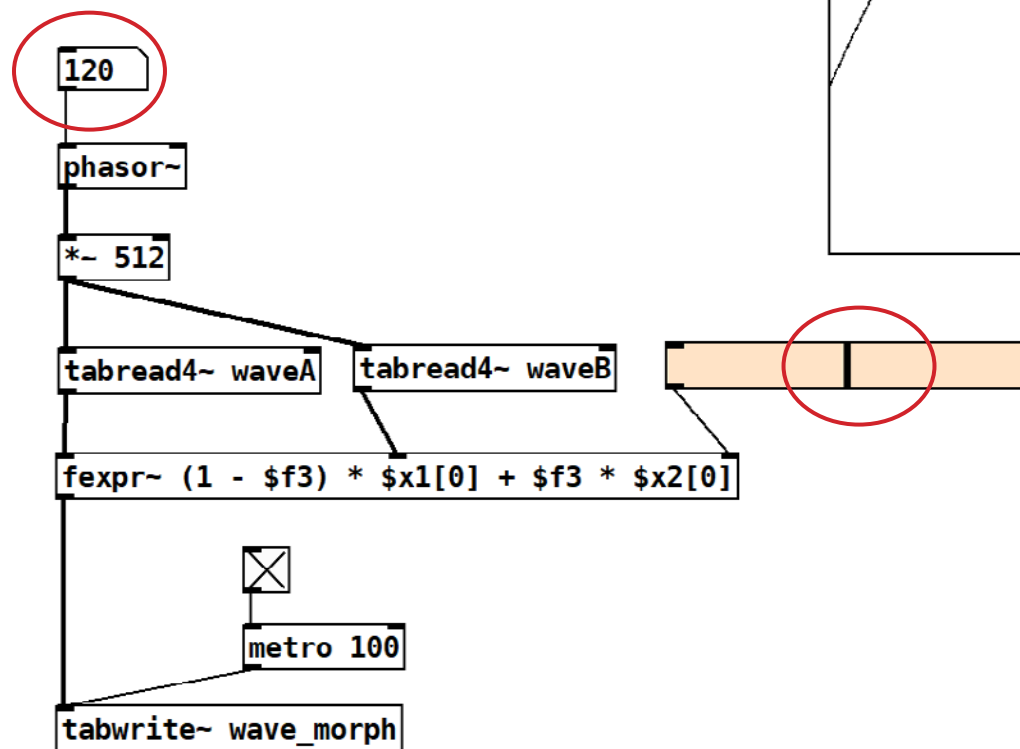
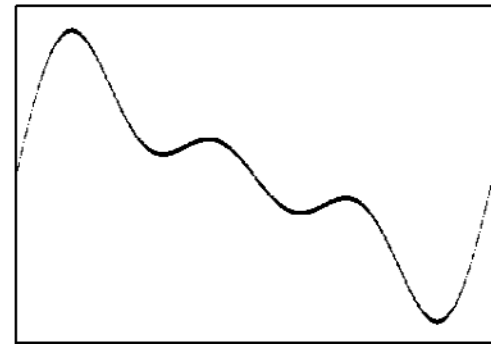


```
;
waveA sinesum 512 1;
waveB sinesum 512 0.5 0.33 0.25
```

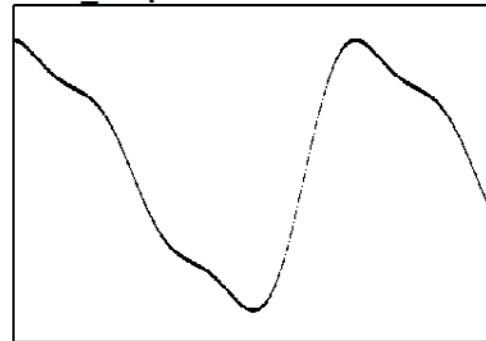
waveA



waveB

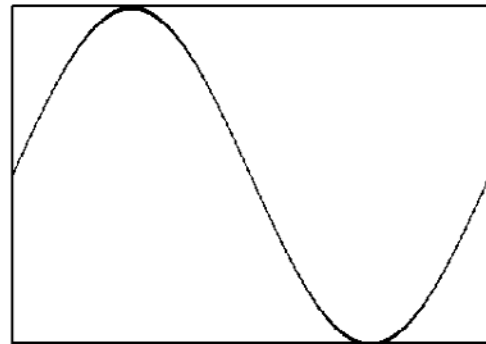


wave_morph

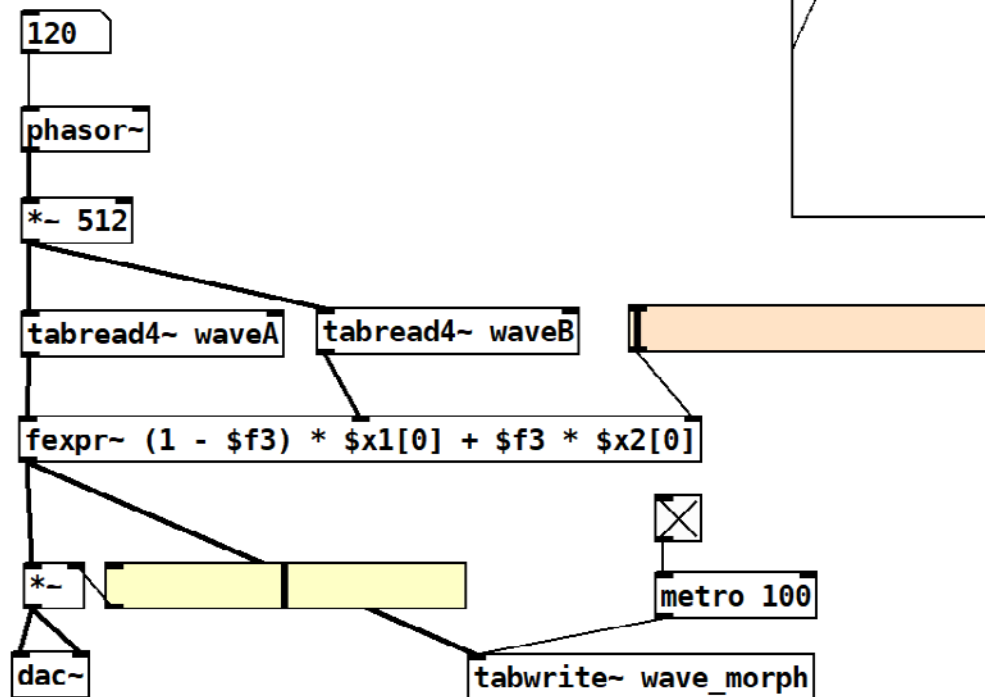
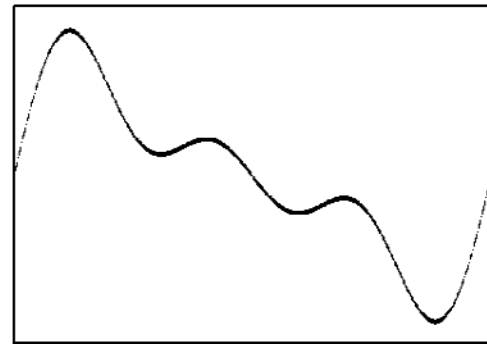


```
;
waveA sinesum 512 1;
waveB sinesum 512 0.5 0.33 0.25
```

waveA



waveB



wave_morph

